



Intellimail

AI Powered Inbox Management Web App

Project Engineering

Year 4

Yi Zheng Tang (G00425075)

Bachelor of Engineering (Honours) in Software and
Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Acknowledgements

I want to thank my project supervisor, Brian O'Shea, for his assistance and guidance throughout the entire project.

I would also like to thank my other supervisors, Ben Kinsella, Paul Lennon, Michelle Lynch, David Newell, Niall Okeefe, as well as my classmates for giving me feedback and support during the development process of the project.

Table of Contents

- 1 Summary 8
- 2 Poster 10
- 3 Introduction 11
- 4 Background & Motivation 12
 - 4.1 Problem Research 12
 - 4.1.1 Declining Attention Spans and Cognitive Overload 12
 - 4.1.2 Excessive Time Spent on Communication Platforms..... 13
 - 4.1.3 Fundamental Limitations of Current Email and Messaging Applications..... 14
 - 4.2 Solution Research 14
 - 4.2.1 AI-Driven Email Management and Productivity Gain 15
 - 4.2.2 Unified Interface Design and Cognitive Load Reduction 15
 - 4.2.3 AI Agents and Intelligent Workflow Automation 16
 - 4.2.4 Retrieval-Augmented Generation and Large Language Models 16
- 5 Project Architecture..... 18
- 6 Technologies Used 19
 - 6.1 React 19
 - 6.2 TypeScript 19
 - 6.3 Vite 19
 - 6.4 Axios 19
 - 6.5 FastAPI 20
 - 6.6 OAuth 2.0 20
 - 6.7 Gmail API..... 21
 - 6.8 Microsoft Graph API..... 21

6.9	OpenAI API	21
6.10	ChromaDB	21
6.11	SQLite	22
7	Project Plan	22
7.1	Project Management Tools	22
7.1.1	GitHub	23
7.1.2	Jira	23
7.1.3	OneNote	24
7.1.4	Development Gantt Chart	25
8	System Implementation	26
8.1	Inbox Provider Integration	26
8.2	Data Ingestion Pipeline	28
8.3	Retrieval Augmented Generation	29
8.4	AI Agents	30
8.4.1	Summarisation Agent	30
8.4.2	Categorisation Agent	30
8.4.3	Prioritisation Agent	30
8.4.4	Event Extraction Agent	30
8.4.5	Reply Generation Agent	30
8.4.6	Spam Detection Agent	30
9	Code Walkthrough	31
9.1	Project Structure	31
9.2	Run.sh	32
9.3	Backend	33

9.3.1	app/main.py.....	34
9.3.2	Gmail Integration	35
9.3.3	Outlook Integration	39
9.3.4	Teams Integration	43
9.3.5	AI Layer	47
9.3.6	Database	53
9.4	Frontend.....	56
9.4.1	Routing and Context Provider.....	57
9.4.2	Type Definitions	58
9.4.3	Services	59
9.4.4	Pages	60
9.4.5	Components.....	60
10	Project Setup.....	61
10.1	Prerequisites	62
10.2	Cloning the repository	62
10.3	Backend Dependencies.....	62
10.4	Frontend Dependencies.....	62
10.5	Google Cloud Console Setup (Gmail).....	62
10.6	Azure App Registration Setup (Outlook and Teams)	63
10.7	OpenAI API Key Setup	64
11	Results.....	65
11.1	Gmail Inbox	65
11.2	Outlook Inbox Page.....	66
11.3	Email Detail Page	67

- 11.4 Unified Inbox Page 68
- 11.5 AI Chat Page 69
- 11.6 Calendar Page 70
- 12 Ethics 71
 - 12.1 Technology Research and Selection 71
 - 12.2 Use of AI as a Development Aid..... 71
 - 12.3 Data Privacy 71
- 13 Conclusion..... 72
- 14 References 73

1 Summary

Individuals today often juggling between multiple communication platforms, personal and institutional email accounts like Gmail and Outlook, messaging applications such as Microsoft Teams and WhatsApp. Which results in fragmented communication and reduced productivity. Constant switching between applications, cluttered inboxes, and buried important messages contribute to information overload, missed deadlines, and poor time management.

Intellimail is an AI-Powered Unified Inbox Management Web App that consolidates Gmail, Outlook, and Microsoft Teams into a single unified interface, acting as a personal intelligent assistant that helps users manage their communications more effectively.

Intellimail was built using React and TypeScript for the frontend, communicating with a Python FastAPI backend. Users connect their accounts through OAuth 2.0 authentication, allowing the application to fetch and manage emails and messages through the Gmail API, Microsoft Graph API, and Microsoft Teams API. All messages are processed through an AI ingestion pipeline that converts content into semantic vectors, stored in a local ChromaDB vector database that serve as the knowledge base. A separate SQLite database manages calendar events extracted from inboxes. When AI features are triggered, relevant data is retrieved from the knowledge base using Retrieval-Augmented Generation (RAG), along with six specialised AI agents responsible for summarising, categorising, prioritising, extracting events, generating replies, and detecting spam. Users can interact with their inbox exactly as they would in Gmail, Outlook, or Teams, with the added capability of AI-powered assistance.

Throughout this project, I developed and enhanced my skills in multiple areas including multi-platform OAuth API integration across Google, Microsoft, and Teams services, vector embeddings and semantic search, and the design and implementation of a RAG framework from scratch.

Intellimail demonstrates a fully functional application that can be used for personal daily communication management, consolidating real accounts and delivering real AI-powered insights on actual inbox data.

2 Poster



INTELLIMAIL
 G00425075
 BENG (HONS) IN SOFTWARE
 & ELECTRONIC ENGINEERING

YI ZHENG TANG

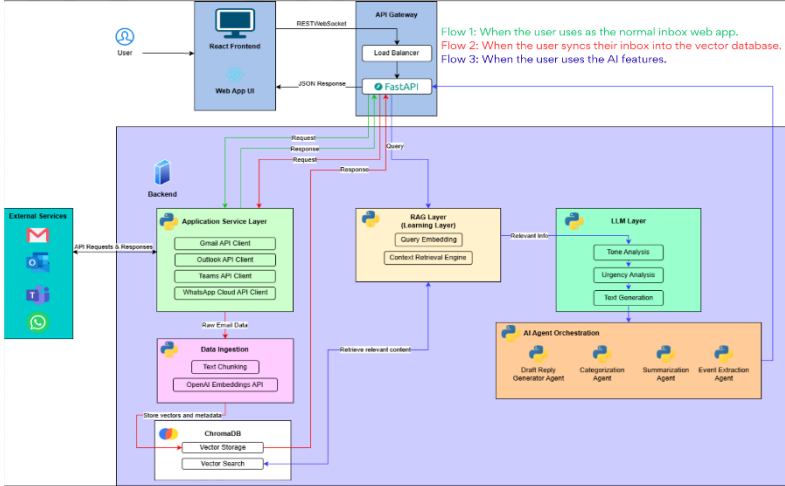
G00425075
BENG (HONS) IN SOFTWARE
& ELECTRONIC ENGINEERING

AI POWERED INBOX MANAGEMENT WEB APP

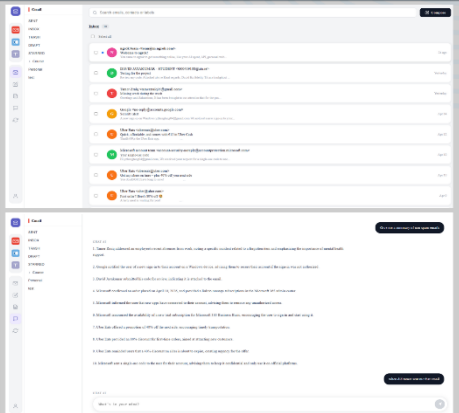
PROBLEM	HOW IT WORKS
Individuals often juggle multiple communication platforms, including personal and institutional/work email accounts, messaging applications such as WhatsApp and Instagram, resulting in fragmented communication and reduced productivity. Constant switching between apps, cluttered inboxes, and hidden important messages contribute to overload, missed deadlines, and poor time management.	• Connect: User links their accounts through secure OAuth authentication. • Unify: All messages from every connected platform are pulled into a single, organized inbox. • Sync: The app syncs the inbox, converting inboxes into a searchable AI knowledge base. • Analyse: Multiple AI Agents automatically summarize, categorize, prioritize and scan for events across all messages/inboxes. • Interact: Users can chat with the AI living in the web app such as asking questions regarding their inboxes, finding emails, or drafting replies.
SOLUTION Intellimail proposes a centralized web app that consolidates multiple email and messaging platforms into a single interface, enhanced with RAG and AI agents that act as a personal assistant that intelligently process, summarize, and respond to messages, in order to enhance productivity.	Using React and Typescript for frontend that communicates with Python backend services, user connects their social accounts through OAuth authentication. This allows the web app to fetch and manage inboxes through the API service layer. The inboxes are processed through an AI ingestion pipeline that converts into semantic vectors that is stored in the vector database. When AI features are triggered, relevant data is retrieved using RAG and passed as context which powers the AI agents that are responsible for different tasks.

ARCHITECTURE DIAGRAM

RESULTS



Flow 1: When the user uses as the normal inbox web app.
 Flow 2: When the user syncs their inbox into the vector database.
 Flow 3: When the user uses the AI features.



A fully working unified inbox web app where user can navigate through different communication platforms, and also an AI chat page.

WHAT IS RAG AND WHY?

TECHNOLOGIES USED

Retrieval Augmented Generation (RAG) is a technique that enables LLMs to retrieve and incorporate new information from a specific data sources. With RAG, LLMs will refer to a specified set of documents, then respond to user queries.

This project uses RAG so that the AI agents can generate responses grounded in the user's actual emails rather than relying solely on GPT's general training data, ensuring the answers are accurate, relevant, and specific to the user's inbox.












SCAN ME

3 Introduction

The main goal of this project is to develop a web application that can make a positive impact on personal productivity by consolidating fragmented communication platforms into a single, united interface. In an era where the rapid growth of artificial intelligence is reshaping how people work and communicate, individuals are increasingly turning to AI-powered tools to assist with everyday tasks, both in professional environments and in their personal lives. At the same time, the modern working style has shifted significantly towards remote and hybrid work, which has deepened reliance on digital communication platforms such as email, instant messaging, and collaboration tools.

This project was also motivated by my personal interest in Artificial Intelligence, specifically in how emerging techniques such as AI Agents and Retrieval Augmented Generation can be applied to solve real, practical problems. This project represents an opportunity to bring together my skills and knowledge into a meaningful final year project. And the main inspiration is that many repetitive and time-consuming daily tasks, can be effectively automated with the thoughtful application of AI.

The scope of this project covers the design and development of a full-stack web application integrating Gmail, Outlook, and Microsoft Teams through their respective official APIs. The system includes a complete AI layer including a full data ingestion pipeline, a vector database for semantic search, a Retrieval-Augmented Generation framework, and six specialised AI agents capable of summarising, categorising, prioritising, extracting events, generating replies, and detecting spam. A calendar feature that automatically populates from extracted events is also included within scope.

This report will document the full development of Intellimail, including the full background research, technologies used, relevant research findings, system architecture, code walkthroughs, ethical considerations, and lastly a conclusion reflecting on what was achieved and the direction for future development.

4 Background & Motivation

This is the “Why before the What” section. This section will provide the contextual foundation that justifies the need for Intellimail. Below is the full research done before the development starts.

All research, findings, and personal reflections documented in this section were independently conducted and recorded in OneNote. Artificial intelligence tools were used solely to assist in rephrasing and structuring the content into a formal report format, and do not represent the source of any research or conclusions presented here.

4.1 Problem Research

4.1.1 Declining Attention Spans and Cognitive Overload

In the current digital era, where individuals spend the majority of their time on electronic devices for both professional and personal purposes, attention spans have undergone a dramatic decline. Research conducted by Dr. Gloria Mark found that the average attention span on screens dropped from approximately 2.5 minutes in 2004 to just 44 to 50 seconds by 2021 [1]. This sharp decline is not merely a behavioural shift; it reflects a deeper cognitive burden imposed by the nature of modern digital work environments.

This issue extends beyond shortened focus periods. When users are required to juggle multiple communication platforms simultaneously, the brain is forced to continuously reload mental context, adapt to different interfaces, and process different communication styles and conventions. Research by Jill Duffy found that when workers switch between tasks, it takes an average of 25 minutes to return to their original task, and individuals typically encounter an average of 2.3 other tasks before resuming their initial work [2]. The cumulative effect of this constant context switching is not only time-consuming but significantly impairs working productivity and cognitive performance.

Developing a unified web application that consolidates all communication platforms into a single dashboard with a consistent interface directly addresses this challenge. By eliminating the need to switch between applications, the cognitive effort required to adapt to different environments is reduced, allowing the user to maintain focus and operate more efficiently.

4.1.2 Excessive Time Spent on Communication Platforms

The rise of digital communication tools such as email, Microsoft Teams, and Slack has introduced a paradox, while these platforms were designed to increase productivity, research suggests they are increasingly doing the opposite. A report by Marketplace found that while digital platforms have increased productivity in certain respects, they have also resulted in significantly more time spent responding to messages and less time dedicated to meaningful productive work [3]. Overwhelmed inboxes and the pressure of constantly managing ongoing conversations contribute to workplace anxiety and negatively impact both productivity and work quality

Email overload is among the most prominent manifestations of this problem. It occurs when the volume of emails an individual receives exceeds their capacity to process and respond to them in a timely and effective manner [4]. The consequences include missed deadlines, ignored messages, and large volumes of unread correspondence. Beyond productivity, email overload introduces security risks, when inboxes are flooded with messages, it becomes significantly more difficult for individuals to identify malicious content such as phishing attempts or malware-infected attachments before engaging with them [4].

This challenge is particularly relevant in the context of the increase in remote and hybrid working arrangements in recent years, which has made organisations increasingly dependent on digital communication platforms. While existing email platforms offer basic organisational tools such as label-based categorisation and keyword search, these features are limited in their intelligence and adaptability. A more capable system that leverages AI for summarisation, automated categorisation, and intelligent prioritisation represents a meaningful improvement over the current state of these tools.

4.1.3 Fundamental Limitations of Current Email and Messaging Applications

Modern email applications suffer from a range of fundamental limitations, including poor search functionality, inadequate prioritisation and filtering mechanisms, and inflexible workflows that do not adapt to individual user behaviour patterns [5]. These shortcomings are particularly significant given that approximately 90% of daily email users check their email every single day, and email remains one of the most relied-upon channels for professional and marketing communication [5].

Research has further found that email is inherently stress-inducing, the longer an individual spends managing email within a given hour, the higher their reported stress levels for that hour [5]. The psychological burden of email management has become so significant that some countries, such as France, have enacted labour laws specifically designed to protect employees' right to disconnect outside of working hours, in an effort to reduce burnout [5].

This suggests that the problem is not simply a matter of improving existing tools, but rather that the current systems are fundamentally insufficient for the demands of modern communication. The volume and complexity of digital communication have grown beyond what rule-based sorting and manual organisation can effectively handle. The introduction of intelligent software, incorporating AI agents, natural language processing, and adaptive learning — represents a necessary evolution toward a communication system that is not only more efficient, but genuinely adaptive to user behaviour.

4.2 Solution Research

Having identified the key problems, here is the full research conducted on the solution approach.

4.2.1 AI-Driven Email Management and Productivity Gain

Research has shown that AI can increase productivity by 20 to 30 percent through the automation and optimization of various working processes, including email management [6]. Beyond the basic sorting mechanisms offered by existing platforms, such as sorting by label, keyword, or subject. AI-driven categorization leverages machine learning algorithms and natural language processing to automatically classify incoming messages into contextually relevant categories, such as work-related correspondence, personal communication, promotional content, spam, and high-priority messages [6]. This level of intelligent automation reduces the manual effort required to maintain an organized inbox and enables users to focus their attention on communications that genuinely require it.

4.2.2 Unified Interface Design and Cognitive Load Reduction

Unified communications is defined as a concept that integrates multiple enterprise communication services into a consistent user interface and experience across multiple devices and media types [7]. By consolidating all communication applications into a single platform, unified systems provide the mobility, contextual awareness, and accessibility that individual siloed applications cannot offer. Research has further shown that unified interfaces provide a more consistent and accessible user experience, and that chat-driven unified interfaces in particular offer a single conversational entry point capable of handling all user interactions, which significantly simplifying the user's experience when dealing with complex systems [8].

These principles directly informed the design of Intellimail, where a single React-based web interface consolidates Gmail, Outlook, and Microsoft Teams with a consistent UI design. This approach reduces the cognitive burden associated with switching between different applications and their respective interfaces, improving both productivity and usability.

4.2.3 AI Agents and Intelligent Workflow Automation

AI agents are autonomous software systems designed to support users by reducing repetitive work, improving decision accuracy, and accelerating task completion. Research has found that organizations that integrate AI agents into their workflows give employees over two additional productive hours per day, leading to measurably higher performance and working efficiency [9]. In the context of communication management, AI agents can take on tasks such as drafting replies, summarising long email threads, detecting spam, and extracting actionable events, all of which are time-consuming when performed manually.

4.2.4 Retrieval-Augmented Generation and Large Language Models

Large Language Models (LLMs) are AI-based computational models trained on vast datasets, designed to understand and generate human-like text. With billions of training parameters, LLMs excel at identifying complex language patterns and performing tasks such as text summarisation, reply generation, and sentiment analysis [10]. However, LLMs in isolation are limited by their static training data, they are unable to reason accurately about domain-specific or up-to-date information, and are prone to generating plausible but incorrect outputs, a phenomenon known as hallucination.

Retrieval-Augmented Generation (RAG) is a framework designed to address these limitations by combining retrieval mechanisms with language generation. Rather than relying solely on pre-trained knowledge, a RAG system retrieves relevant documents from an external knowledge base and conditions the language model's output on that retrieved content [10]. In the context of Intellimail, the user's own emails and messages serve as the knowledge base. When an AI agent is triggered, semantically relevant emails are retrieved from the vector database and passed as context to the language model, grounding its output in the user's actual communication history.

This approach significantly enhances the accuracy and relevance of AI-generated outputs, addressing the core limitations of standalone LLMs. Studies have shown that RAG-powered tools reduce output errors by up to 15% compared to traditional AI systems operating without retrieval [11]. Furthermore, RAG enables continuous knowledge updates without retraining the underlying model, allowing the system to remain current as the user's inbox evolves and to learn from domain-specific patterns such as individual communication style and recurring correspondents [12].

5 Project Architecture

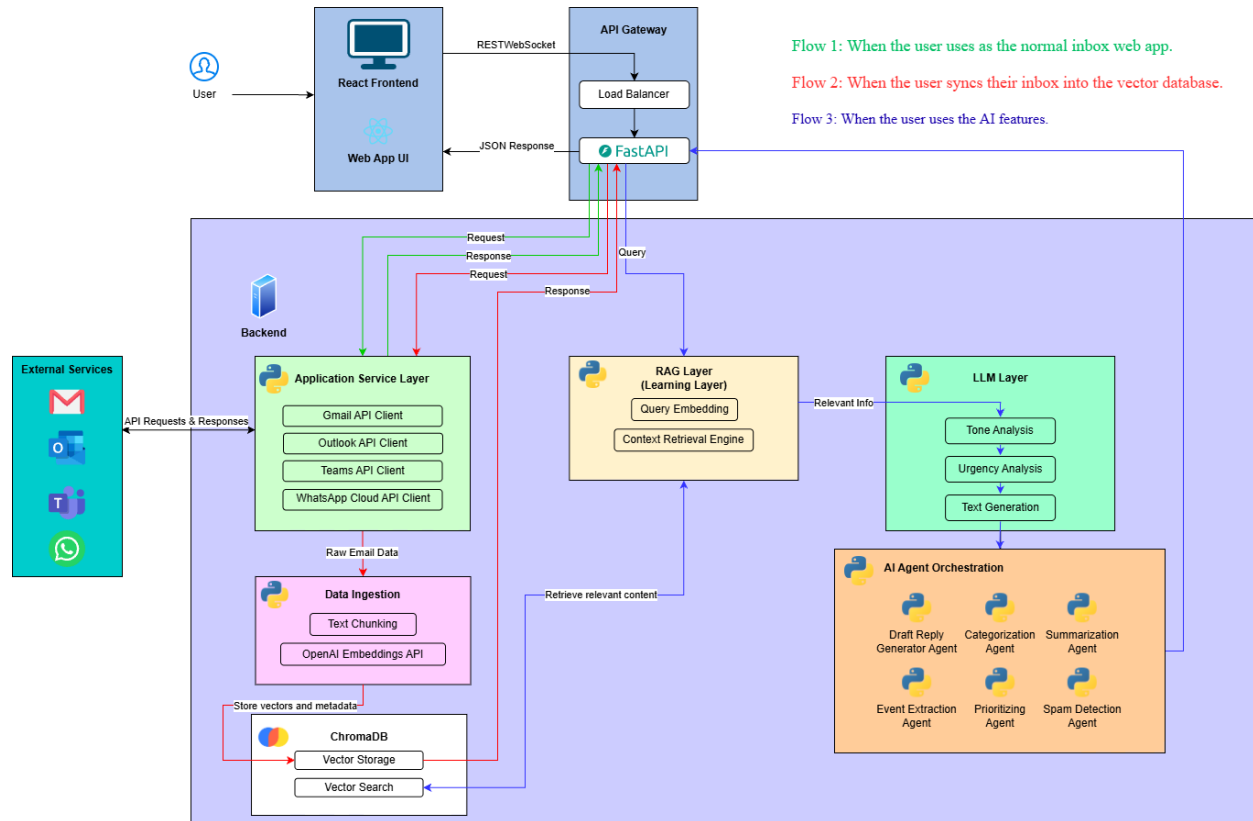


Figure 5-1 Architecture Diagram

The system follows a layered architecture where the user interacts with a React-based frontend, which communicates with a FastAPI backend through a REST API gateway. The backend is divided into four core layers. The **Application Service Layer** handles all communication with external providers like Gmail, Outlook, Microsoft Teams, and WhatsApp through their respective API clients. Raw email data retrieved from these providers is passed to the **Data Ingestion Layer**, where it is chunked and embedded using the OpenAI Embeddings API before being stored as vectors in ChromaDB. When an AI operation is triggered, the **RAG Layer** queries ChromaDB to retrieve contextually relevant content, which is then forwarded to the **LLM Layer** for tone analysis, urgency analysis, and text generation. Finally, the **AI Agent Orchestration Layer** coordinates six specialised agents, each consuming the LLM output to perform their respective tasks and return structured results to the frontend.

6 Technologies Used

6.1 React

React is an open-source JavaScript library developed by Meta for building user interfaces through a reusable, component-based architecture [13]. It was chosen for Intellimail because its component model is the perfect fit for a web application structure, having multiple pages, different sidebar view and panels, they all exist as isolated components that can be developed and maintained independently. React's virtual DOM ensures efficient re-rendering without reloading the entire page.

6.2 TypeScript

TypeScript is a statically typed superset of JavaScript developed by Microsoft that compiles down to plain JavaScript [14]. It was used throughout the entire frontend codebase to enforce strict type checking on all component props, API responses, and service functions. This eliminated a large class of potential runtime errors during development, as mismatches between what the backend returns and what the frontend expects are caught at compile time rather than in the browser.

6.3 Vite

Vite is a modern frontend build tool that serves as the development server and bundler for the React application [15]. It was selected over the older Create React App toolchain due to its significantly faster cold start time and near-instant hot module replacement, which made iterative UI development considerably more efficient.

6.4 Axios

Axios is an HTTP client library used to make all API requests from the frontend to the FastAPI backend [16]. A single configured Axios instance is shared across all service files, with the backend base URL and a centralised error response interceptor defined in one place, meaning no individual service function needs to handle errors or configure headers manually.

6.5 FastAPI

FastAPI is a modern Python web framework used to build the entire backend API [17]. It was chosen over alternatives such as Flask and Django because it natively supports asynchronous request handling, enforces request and response validation through Pydantic models, and automatically generates interactive API documentation through Swagger UI. The backend is structured as a collection of routers, one per provider and one for the AI layer, and each are registered with the central FastAPI application.

6.6 OAuth 2.0

OAuth 2.0 is the industry-standard authorization protocol used to connect user accounts from Gmail, Outlook, and Microsoft Teams without requiring the user to share their credentials directly with Intellimail [18]. Gmail authentication is handled through Google's OAuth 2.0 flow using the **google-auth** and **google-auth-oauthlib** Python libraries, while Outlook and Teams use the Microsoft Authentication Library (MSAL). Both flows use a local callback server, so the full authorization exchange happens automatically.

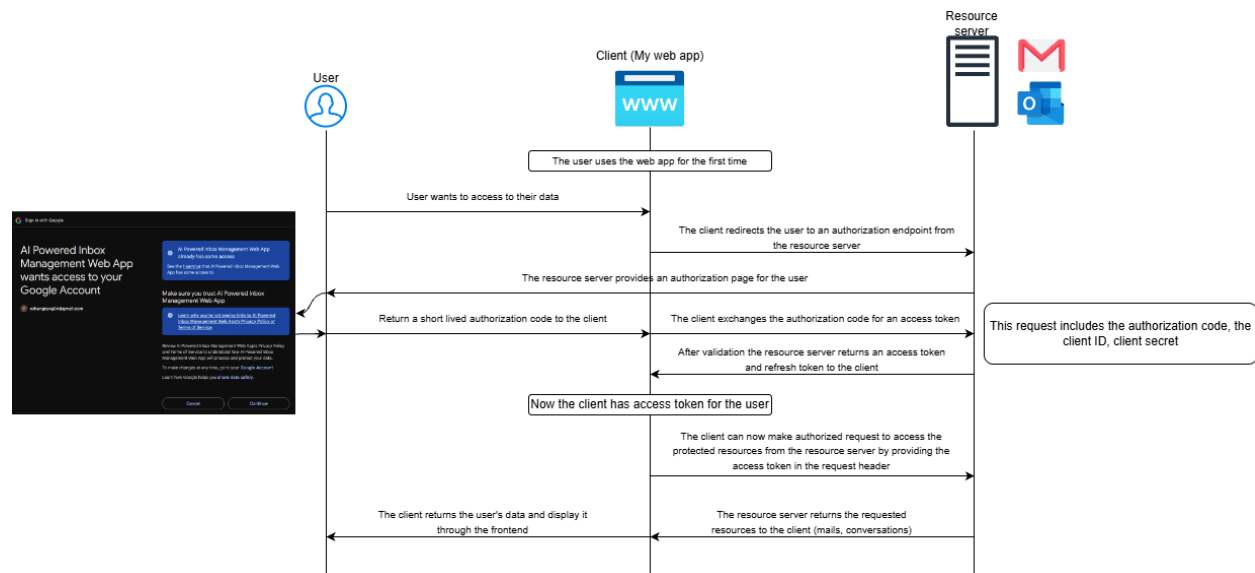


Figure 6-1 OAuth Sequence Diagram

6.7 Gmail API

The Gmail API is Google's official REST API for accessing Gmail inbox data [19]. It is accessed through the Google API Python Client library, which wraps the underlying HTTP calls into a structured service object. It is used in Intellimail to perform inbox CRUD operations within a connected Gmail account.

6.8 Microsoft Graph API

The Microsoft Graph API is Microsoft's unified REST API that provides access to data across Microsoft 365 services, including Outlook email and Microsoft Teams [20]. It is called directly via the **httpx** HTTP client in the backend. It is used in Intellimail to perform inbox CRUD operations, as well as fetching Microsoft Teams chat messages.

6.9 OpenAI API

The OpenAI API provides access to two models used in Intellimail [21]. The **text-embedding-3-small** model converts email and message content into 1536-dimensional semantic vectors, which are stored in ChromaDB and used for similarity search during RAG retrieval. The **gpt-4o-mini** model powers all six AI agents. GPT-4o-mini was selected for its 128,000 token context window, strong instruction-following capability, and low-cost relative to larger models.

6.10 ChromaDB

ChromaDB is an open-source vector database used to store and query email embeddings [22]. It runs as a local PersistentClient, saving all data to disk without requiring any external service, API key, or Docker setup. All emails from all providers are stored in a single unified collection, with the provider identified through metadata. Cosine similarity is used as the distance metric because text embeddings encode semantic meaning as the direction of a vector, making cosine the correct measure for comparing the meaning of two pieces of text.

6.11 SQLite

SQLite is a lightweight, self-contained relational database engine used to store structured calendar events extracted from emails [23]. It was used separately from ChromaDB because calendar data requires a relational database instead of a vector database. SQLite is accessed directly through Python's built-in **sqlite3** module with no ORM layer.

7 Project Plan

Intellimail was developed using an iterative development approach, basically the project was broken down into multiple phases, each building directly on what was done by the previous one. Each phase focused on delivering a full working vertical slice of the application before the next phase began. This approach allowed problems to be identified and resolved early, and ensured the system remained functional and testable throughout the entire development period.

Within each phase, development followed a strict commit discipline, no half-finished features were committed, and no feature was considered complete until it had been pushed to the remote repository. This granular approach kept the codebase clean and made it straightforward to trace every change back to a specific decision.

7.1 Project Management Tools

Three tools were used to plan, track, and document the development of Intellimail throughout the project lifecycle.

7.1.1 GitHub

GitHub was used for version control and code management. All development was carried out on dedicated feature branches named after the feature being implemented. Once a feature was complete, a pull request was opened against the development branch for review before merging.



Figure 7 - 1 GitHub Version Control

7.1.2 Jira

Jira was used as the primary project management tool for tracking tasks, phases, and progress. Each development phase was represented as an epic, with individual tasks broken down into tickets. This provided a structured view of what had been completed, what was in progress, and what remained, helping to keep development on schedule throughout the academic year.

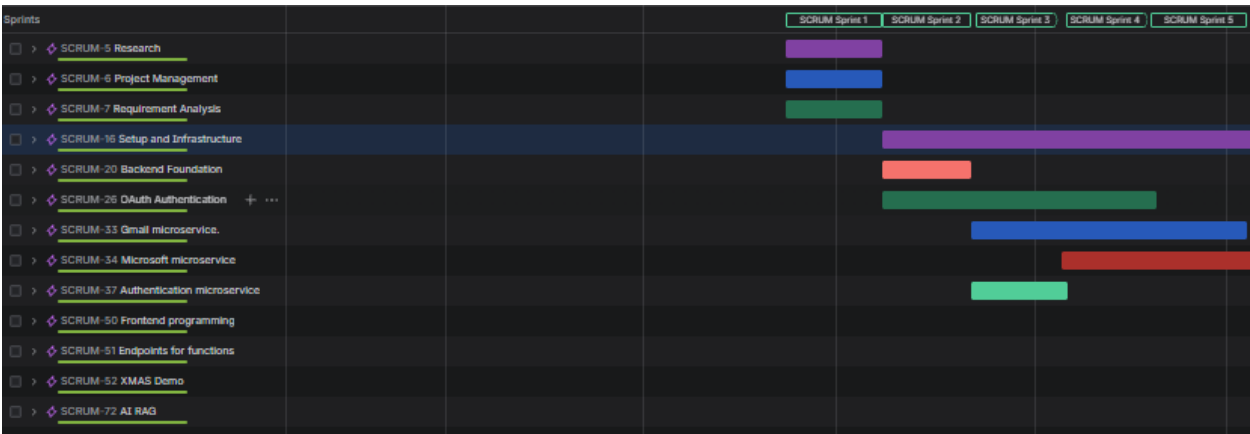


Figure 7 - 2 Jira Timeline and Epics

7.1.3 OneNote

Microsoft OneNote was used to maintain a weekly development log throughout the project. After each significant development step such as completing a feature, solving a problem, or researching a new concept, a log entry was written recording what was planned, what was built, what problems were encountered, and how they were resolved. This diary formed a continuous record of the decision-making process across the full development timeline.

Week 12

Tuesday, 7 April, 2026 7:48 PM

The first thing I do this week is to add a new service to the project, which is the Microsoft Teams.

I did some research and found out that the Microsoft Teams we using in school will require admin access to allow third parties to gain access to it, because it is an organization account. So I have to create a personal Microsoft Teams account to make it work.

The ~~implementation~~ will be pretty much straight forward just like what I have for the Gmail and Outlook. And since I am using the same Microsoft Azure App registry, the first thing I did is to add few API permissions to my app:

Microsoft Graph (3)					...
Chat.ReadWrite	Delegated	Read and write user chat messages	No		...
Team.ReadBasic.All	Delegated	Read the names and descriptions of teams	No		...

I added these two ~~api~~ permission to the app, and I can use them to perform CRUD operations for Teams.

Then I proceeded to the first implementation for the service, which is the ~~oauth~~ process. I gone through the official Microsoft API documentation and I found out that they are following the same exact authentication process, so then I followed the exact same MSAL OAuth2 pattern that is already built for Outlook, which using the same local server ~~callback~~ and also same refresh token caching method, same ~~ConfidentialClientApplication~~

```
1 #teams_auth.py
2 #MSAL OAuth2 authentication for Microsoft Teams via Microsoft Graph API.
3
4 import os
5 import webbrowser
6 import threading
7 from urllib.parse import urlparse, parse_qs
8 from http.server import HTTPServer, BaseHTTPRequestHandler
9 import msal
10 from dotenv import load_dotenv
11
12 load_dotenv()
13
14 MS_GRAPH_BASE_ENDPOINT = 'https://graph.microsoft.com/v1.0/'
15
16 #Teams stores its refresh token separately from Outlook.
17 #Outlook token has Mail.ReadWrite scope, whereas Teams token has Chat.ReadWrite scope.
18 REFRESH_TOKEN_PATH = os.path.join(os.path.dirname(__file__), 'refresh_token.txt')
19
```

The first part of the script contains the ~~refresh_token_path~~, it is pointed to Teams/refresh_token.txt, it must be separated with the refresh_token.txt for Outlook, because the refresh token for outlook has ~~Mail.readWrite~~ scope that

Figure 7 - 3 OneNote Weekly Logs example

7.1.4 Development Gantt Chart

The project was planned across six phases spanning from October 2025 to May 2026, as illustrated in the Gantt chart below.

Planning and Foundation (October 2025) covered initial research, project scoping, technology selection, and setting up the development environment including GitHub, Jira, and the project repository structure.

Backend Foundation and API Integration (November – December 2025) covered the full implementation of the Gmail and Outlook API integrations, including OAuth 2.0 authentication flows for both providers, and all email management endpoints exposed through the FastAPI backend.

AI Integration and Fine Tuning (December 2025 – February 2026) covered the full AI layer that includes the OpenAI embedding pipeline, ChromaDB vector store, RAG retrieval framework, and all six AI agents. Microsoft Teams API integration was also completed during this phase.

Frontend Development (February – March 2026) covered the complete React and TypeScript frontend, including the unified inbox interface, email detail view, compose and reply flows, folder and label navigation, and the calendar feature.

Testing and Documentation (March – April 2026) covered end-to-end testing of all features across providers, bug fixing, and the preparation of supporting documentation.

Finalisation, Report Writing and Submission (April – May 2026) covers the writing of this report and the final preparation of all project deliverables for submission.

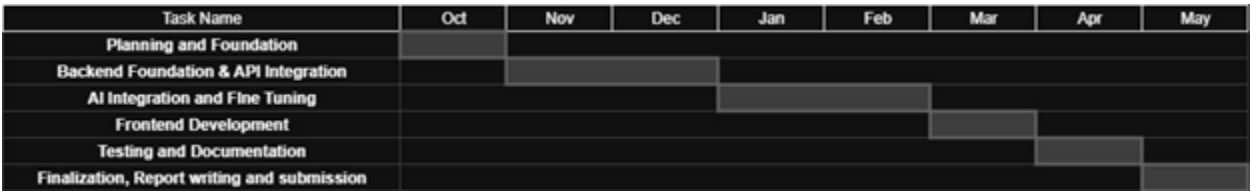


Figure 7 – 4 Project Development Gantt Chart

8 System Implementation

This section provides a high-level overview of how the core features of Intellimail were implemented and how the different parts of the system work together to deliver a unified, AI-powered inbox experience.

8.1 Inbox Provider Integration

The starting point of Intellimail is connecting to the user's accounts. Users authenticate their Gmail and Outlook accounts through OAuth 2.0, an industry-standard authorisation protocol that allows the application to access their inbox data without ever handling their passwords directly. Once authenticated, the application can fetch, read, send, reply to, and manage emails on the user's behalf through the official Gmail API and Microsoft Graph API respectively. Microsoft Teams messages are also fetched through the Microsoft Graph API using the same authenticated connection established for Outlook.

The diagram below shows one of the OAuth examples for Gmail. When a user connects their Gmail account, the browser redirects them to Google's login page, passing the application's client ID to identify Intellimail as the requesting application. Once the user grants permission, Google redirects them back to the application with a short-lived authorization code. The backend server then exchanges this code, along with the client ID and client secret, directly with Google to receive an access token and a refresh token. From that point forward, the backend uses the access token to make authenticated requests to the Gmail API on the user's behalf, fetching emails, sending messages, and managing labels without ever requiring the user to log in again [18].

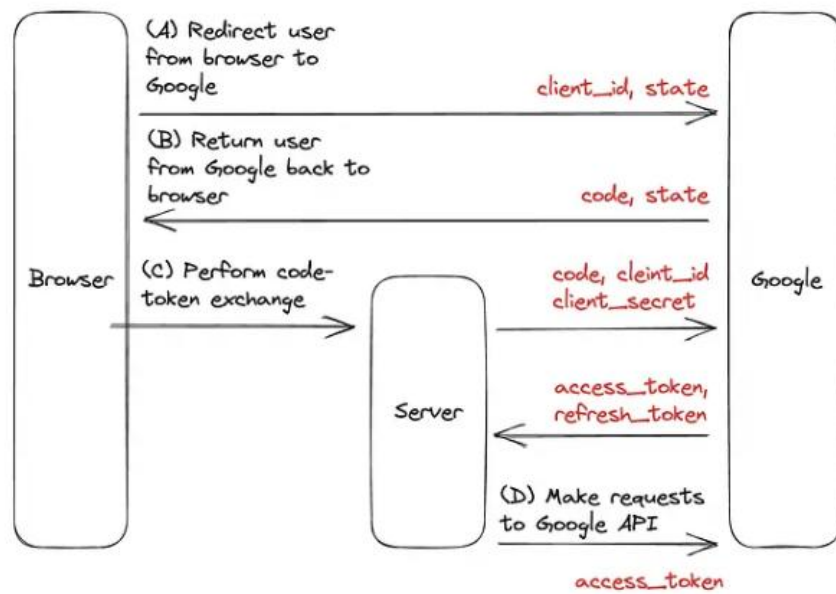


Figure 8 - 1 Google OAuth Example

8.2 Data Ingestion Pipeline

For the AI features to work effectively, the content of the user's emails must first be processed and stored in a way that supports intelligent search. This is done through a data ingestion pipeline that the user can trigger at any time to index their inbox.

During ingestion, each email's text content is extracted and cleaned, then converted into a numerical representation that captures its semantic meaning. This vector embedding is generated by an AI model and stored in a vector database alongside the email's metadata such as the sender, date, and provider. The result is a searchable knowledge base built entirely from the user's own inbox, which the AI agents can query when generating responses. This is what allows Intellimail's AI features to be grounded in the user's actual communication history rather than producing generic outputs.

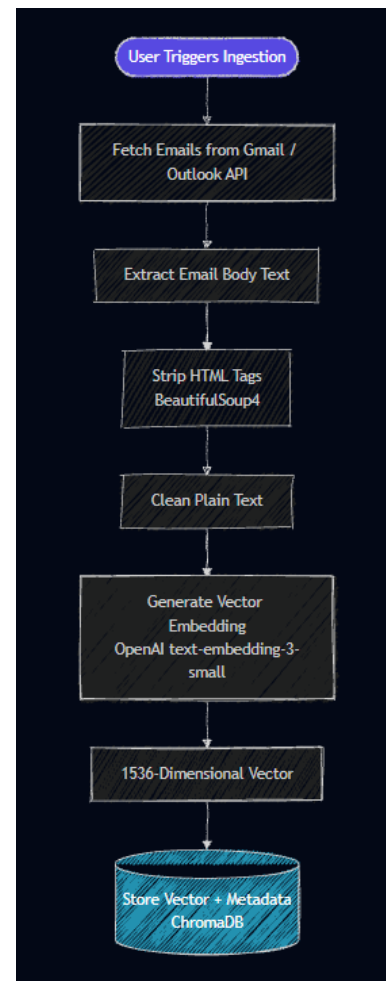


Figure 8 - 2 Ingestion Pipeline

8.3 Retrieval Augmented Generation

When a user sends a query in the AI chat, the system does not simply pass that message directly to the language model. Instead, it first searches the vector database for emails that are semantically related to what the user asked. This retrieved content is then included alongside the user's original message when the AI generates its response [10].

This approach, known as Retrieval-Augmented Generation, significantly improves the quality and relevance of AI outputs. Because the language model receives relevant emails from the user's own inbox as context, its responses are grounded in the user's actual communication history rather than relying solely on its pre-trained knowledge. For example, if a user asks, "What did John say about the project deadline?", the system retrieves emails from John that mention deadlines and passes them to the language model, allowing it to give a specific and accurate answer.

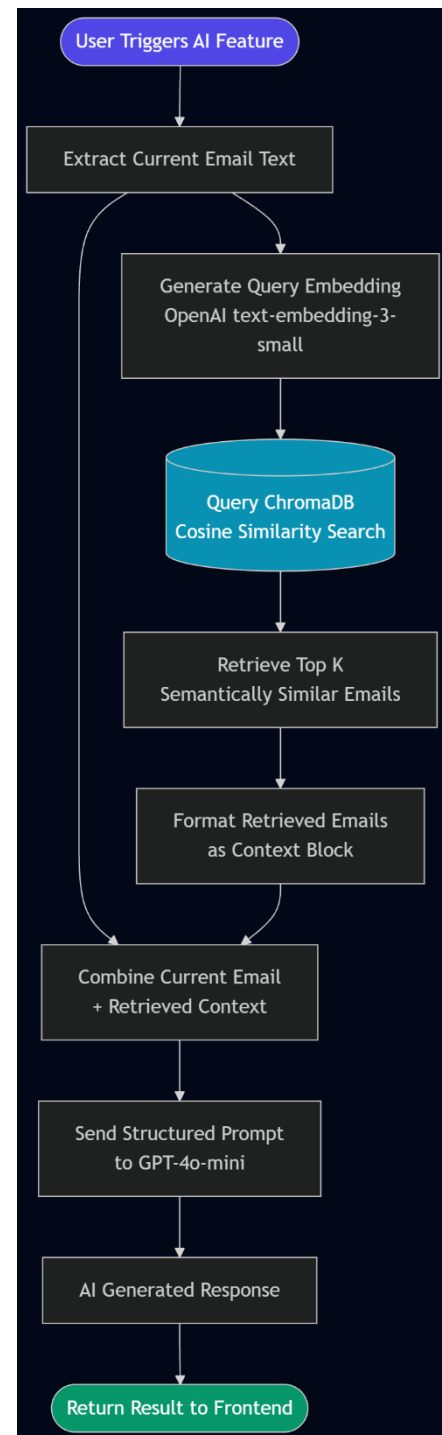


Figure 8 - 3 RAG Pipeline

8.4 AI Agents

Intellimail includes six specialised AI agents, each responsible for a specific task. When triggered, each agent receives the content of the current email along with any relevant context retrieved from the knowledge base and uses a large language model to produce a structured output.

Each agent operates independently and can be triggered individually from the email detail view, giving the user full control over which AI assistance they want to use and when.

The six agents are:

8.4.1 Summarisation Agent

This agent produces a concise summary of the email's key points.

8.4.2 Categorisation Agent

This agent classifies the email into a relevant category such as work, personal, or promotional.

8.4.3 Prioritisation Agent

This agent scores the email by urgency and importance.

8.4.4 Event Extraction Agent

This agent identifies any dates, deadlines, or meetings mentioned in the email.

8.4.5 Reply Generation Agent

This agent drafts a professional response.

8.4.6 Spam Detection Agent

This agent determines whether the email is likely to be malicious or unwanted.

9 Code Walkthrough

This section walks through the key parts of the Intellimail codebase, explaining how each component is designed, why it is structured the way it is, and how it connects to the rest of the system. Rather than covering every file in full, this section focuses on the most significant functions and design decisions across the backend, AI layer, and frontend. Where major challenges were encountered during development, these are documented alongside the solution that was applied.

Due to the number of code and functions, and some of them might be repeated, only necessary part of code will be explained during this section.

For the service endpoint functions, only the four main CRUD operation functions will be explained in detail.

9.1 Project Structure

The project is divided into two top-level directories and a shell script at the root. The backend/ directory contains the entire Python server, covering all provider integrations, the AI layer, and the database. The frontend/ directory contains the full React and TypeScript application. At the root, run.sh is a bash script used to start the application.

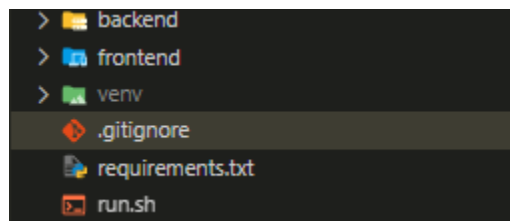


Figure 9.1 – 1 Project Structure

9.2 Run.sh

run.sh is a bash script at the root of the project that provides a simple command interface for starting the application. It accepts a single argument and uses a case statement to determine which server to start.

Running `./run.sh run` starts the FastAPI backend on port 8000 with hot reload enabled, meaning the server automatically restarts whenever a Python file is changed during development.

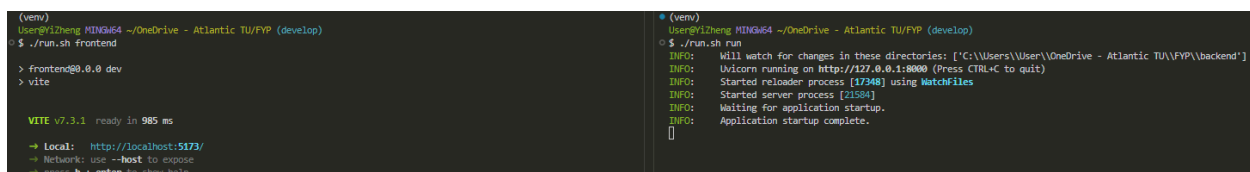
Running `./run.sh start` starts the backend without hot reload, and `./run.sh frontend` starts the Vite development server. If no valid argument is provided, the script prints a usage guide to the terminal.

```

1  #!/bin/bash
2
3  case "$1" in
4      run)
5          cd backend && uvicorn app.main:app --reload --port 8000
6          ;;
7      start)
8          cd backend && uvicorn app.main:app --port 8000
9          ;;
10     frontend)
11         cd frontend && npm run dev
12         ;;
13     *)
14         echo "Run script for fastAPI"
15         echo "  1. run (with reload)"
16         echo "  2. start"
17         echo "  3. frontend"
18         ;;
19     esac
20

```

Figure 9.2 – 1 run.sh



```

(venv)
User@YiZheng MINGW64 ~/OneDrive - Atlantic TU/FYP (develop)
$ ./run.sh frontend
> frontend@0.0.0 dev
> vite

VITE v7.3.1 ready in 985 ms
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

(venv)
User@YiZheng MINGW64 ~/OneDrive - Atlantic TU/FYP (develop)
$ ./run.sh run
INFO: Will watch for changes in these directories: ['C:\\Users\\User\\OneDrive - Atlantic TU\\FYP\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [17348] using WatchFiles
INFO: Started server process [21584]
INFO: Waiting for application startup.
INFO: Application startup complete.

```

Figure 9.2 – 2 Terminal example for starting backend & frontend using bash script

9.3 Backend

The backend is a Python application built with FastAPI, structured into four distinct areas of responsibility.

The app/ folder serves as the entry point and contains the FastAPI application instance and all API routers.

The Gmail/, Outlook/, and Teams/ folders each handle authentication and all API functions for their respective provider.

The ai/ folder contains the full AI pipeline.

The database/ folder manages the SQLite calendar database. The following subsections walk through each of these areas in detail.

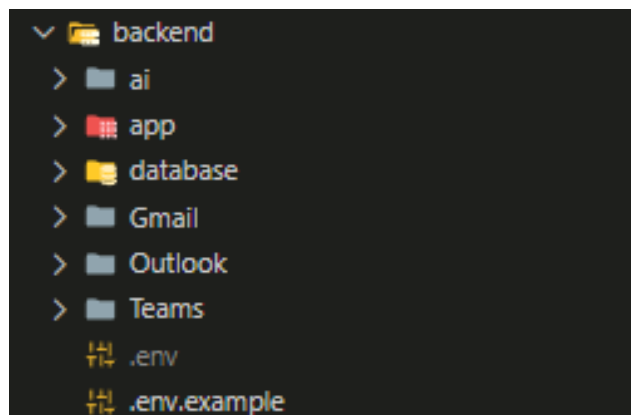


Figure 9.3 – 1 Backend project structure

9.3.1 app/main.py

main.py is the entry point of the FastAPI backend. It creates the FastAPI application instance and registers CORS middleware that allows the frontend running on port 5173 to make requests to the backend on port 8000. Without this middleware, the browser would block all cross-origin requests from the frontend. Each provider and feature area is implemented as a separate router and registered here with its own URL prefix, meaning all Gmail endpoints are served under /gmail, all Outlook endpoints under /outlook, all AI endpoints under /ai, and so on. This keeps each area of the backend self-contained and makes the API structure immediately clear from a single file. Finally, `init_db()` is called at startup to ensure the SQLite events table exists before any calendar requests are made.

```
app = FastAPI(  
    title="AI Inbox Manager",  
    description="Multi-platform inbox with AI-powered categorization",  
    version="1.0.0"  
)  
  
# CORS - Allow frontend to connect (when we build it later)  
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["http://localhost:5173"], # Frontend URL  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)  
  
# Routers  
app.include_router(gmail.router, prefix="/gmail", tags=["Gmail"])  
app.include_router(outlook.router, prefix="/outlook", tags=["Outlook"])  
app.include_router(ai.router, prefix="/ai", tags=["AI"])  
app.include_router(teams.router, prefix="/teams", tags=["Teams"])  
app.include_router(auth.router, prefix="/auth", tags=["Auth"])  
app.include_router(calendar.router, prefix="/events", tags=["Calendar"])  
  
init_db()
```

Figure 9.3.1 app/main.py

9.3.2 Gmail Integration

The Gmail integration is split across two files. `Google_API.py` handles OAuth 2.0 authentication and returns an authenticated service object. `gmail_api.py` contains all functions that interact with the Gmail API using that service object. Every Gmail function takes the service object as its first parameter, keeping authentication completely separate from the API logic.

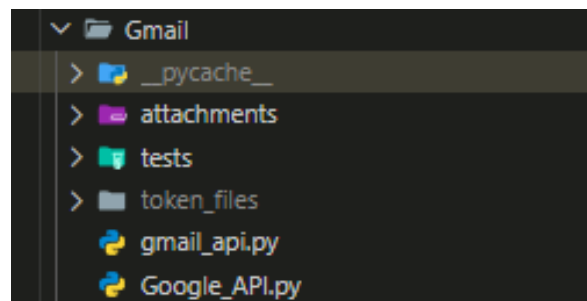


Figure 9.3.2 Gmail folder structure

9.3.2.1 Gmail service initializing

`initialize_gmail_service()` is the entry point for all Gmail operations in Intellimail. It calls `create_gmail_service()` from `Google_API.py`, which handles the full OAuth 2.0 flow, checking for an existing token, refreshing it if expired, or launching the browser-based consent flow if no token exists. The scope `https://mail.google.com/` grants full access to the user's Gmail account, which is required to perform read, send, and delete operations. The returned service object is a dynamically generated Python object that exposes all Gmail API endpoints as method calls and is passed into every other Gmail function.

```
def initialize_gmail_service(api_name = 'gmail', api_version = 'v1', scopes = ['https://mail.google.com/']):  
    service = create_gmail_service(api_name, api_version, scopes)  
    return service
```

Figure 9.3.2.1 Initialize_gmail_service function

9.3.2.2 Get Email Message Details

`get_email_message_details()` retrieves the full content of a single email by its message ID. It calls the Gmail API with `format='full'` to receive the complete message payload including headers and body. The headers are a flat list of key-value pairs, so Python's `next()` with a generator expression is used to extract specific fields such as subject, sender, and date by matching the header name case-insensitively. The body is extracted using a separate `extract_body()` helper function, which recursively traverses the MIME tree of the email to locate the text content, preferring HTML over plain text so that rich emails render correctly in the frontend.

```
#This function will retrieve the full details of a specific email.
#Takes in message_id as a parameter to identify the email to fetch.
def get_email_message_details(service, message_id, user_id='me'):
    #Use the Gmail API to get the full details of a specific email message using its unique message ID.
    #Using the provided message_id and format 'full' to get all details of the email.
    message = service.users().messages().get(user_id=user_id, id=message_id, format='full').execute()

    #From the response, extract the payload from the message and retrieve the headers.
    #Headers contain important metadata such as subject, sender, recipient, date.
    payload = message['payload']
    headers = payload.get('headers', [])

    #Here use list comprehension to find the header with the name "subject" and extract its value.
    subject = next((header['value'] for header in headers if header['name'].lower() == 'subject'), None)
    if not subject:
        subject = message.get('subject', 'No subject')

    #Extract all other metadata with similar approach here.
    sender = next((header['value'] for header in headers if header['name'].lower() == 'from'), 'Unknown sender')
    recipient = next((header['value'] for header in headers if header['name'].lower() == 'to'), 'Unknown recipient(s)')
    snippet = message.get('snippet', 'No snippet available')
    thread_id = message.get('threadId', message_id)
    has_attachments = any(part.get('filename') for part in payload.get('parts', []) if part.get('filename'))
    date = next((header['value'] for header in headers if header['name'].lower() == 'date'), 'No date available')
    star = message.get('labelIds', []).count('STARRED') > 0
    label = ' , '.join(message.get('labelIds', []))

    #Using the extract_body function to get the body content and type from the email payload.
    body, body_type = extract_body(payload)

    email_details = {
        'id': message_id,
        'subject': subject,
        'from': sender,
        'to': recipient,
        'snippet': snippet,
        'thread_id': thread_id,
        'body': body,
        'body_type': body_type,
        'has_attachments': has_attachments,
        'date': date,
        'starred': star,
        'label': label
    }
    return email_details
```

Figure 9.3.2.2 `get_email_message_details` function

9.3.2.3 Sending Email with Attachments

`send_email_with_attachment()` constructs and sends an email through the Gmail API. The Gmail API requires the email to be formatted as a MIME message and encoded as a Base64 URL-safe string rather than a plain JSON body. A `MIMEMultipart` object holds the email body and any file attachments, each encoded in Base64, before the entire message is serialised and sent via the `messages.send` endpoint.

```
#This function will send an email with attachments.
def send_email_with_attachment(service, to, subject, body, body_type='plain', attachment_paths=None):
    #First create the email, by creating a MIMEMultipart object to represent the email message that is about to send.
    message = MIMEMultipart()
    message['to'] = to
    message['subject'] = subject

    #Validates the body_type input, ensures that it is either plain or HTML.
    if body_type.lower() not in ['plain', 'html']:
        raise ValueError("body_type must be either 'plain' or 'html'")

    #Attach the email body to the message object using MIMEText, along by specifying the body type.
    message.attach(MIMEText(body, body_type.lower()))

    #This block will check for the attachment paths provided.
    if attachment_paths:
        #Iterate through each attachment path to attach files to the email.
        for attachment_path in attachment_paths:
            #For each attachment found, first check if the file exists.
            if os.path.exists(attachment_path):
                #Extract the filename.
                filename = os.path.basename(attachment_path)

                #Open the file in binary read mode (rb) to read it's content.
                with open(attachment_path, 'rb') as attachment_file:
                    #Creates a MIME part with type application/octet-stream, which is a generic binary file type.
                    part = MIMEBase('application', 'octet-stream')
                    #Reads the entire file and set it as the payload.
                    part.set_payload(attachment_file.read())

                    #Next step is to encode the payload using Base64 encoding.
                    #This ensures the data remains safe during email transmission.
                    encoders.encode_base64(part)

                    #Then add a header to the attachment part to indicate it's an attachment with the filename.
                    part.add_header("Content-Disposition", f'attachment; filename="{filename}"')

                    #After everything attach the part to the main message object.
                    message.attach(part)
            else:
                raise FileNotFoundError(f"Attachment file '{attachment_path}' not found.")

    #First encode the entire message as a url safe base64 string.
    raw_message = base64.urlsafe_b64encode(message.as_bytes()).decode('utf-8')

    #Use the Gmail API to send the email message.
    sent_message = service.users().messages().send(
        userId='me',
        body={'raw': raw_message}
    ).execute()

    return sent_message
```

Figure 9.3.2.3 `send_email_with_attachments` function

9.3.2.4 Modifying labels

`modify_email_labels()` adds or removes Gmail labels from an email, and is the function that powers mark as read and mark as unread in Intellimail. In Gmail, read state is controlled entirely through labels — marking an email as read simply removes the UNREAD label. Labels are processed in batches of 100 to stay within the Gmail API's per-request limit.

```
#This function will add/delete labels from a specific email message.
def modify_email_labels(service, user_id, message_id, add_labels = None, remove_labels = None):
    #this function will process labels in batches to avoid exceeding API limits.
    def batch_labels(labels, batch_size = 100):
        return [labels[i:i + batch_size] for i in range(0, len(labels), batch_size)]
    if add_labels:
        for batch in batch_labels(add_labels):
            service.users().messages().modify(
                userId=user_id,
                id=message_id,
                body={'addLabelIds': batch}
            ).execute()

    if remove_labels:
        for batch in batch_labels(remove_labels):
            service.users().messages().modify(
                userId=user_id,
                id=message_id,
                body={'removeLabelIds': batch}
            ).execute()

    return f'Labels modified for message ID {message_id}.'
```

Figure 9.3.2.4 `modify_email_labels` function

9.3.2.5 Deleting Email

This is the DELETE operation that deletes a specific email from the Gmail TRASH folder using the `messages.delete` endpoint from Gmail API.

```
#This function permanently deletes an email.
def delete_email(service, user_id, message_id):
    service.users().messages().delete(userId=user_id, id=message_id).execute()
    return f'Message ID {message_id} deleted successfully.'
```

Figure 9.3.2.5 `delete_email` function

9.3.3 Outlook Integration

The Outlook integration follows the same structure as Gmail. MS_Graph.py handles OAuth 2.0 authentication and outlook_api.py contains all API functions.

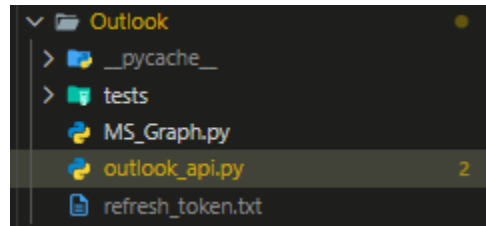


Figure 9.3.3 Outlook folder structure

9.3.3.1 Graph Request Helper Function

make_graph_request() is a shared helper that handles all HTTP communication with the Microsoft Graph API. Every Outlook function calls this helper rather than making raw HTTP requests directly. This is the keep difference with Gmail, where there are built in methods available from the service object. It constructs the Authorization header using the Bearer token, builds the full URL from the base endpoint and the provided path, and dispatches the correct HTTP method.

```
def make_graph_request(access_token, endpoint, method='GET', params=None, json_data=None, extra_headers=None):
    headers = {
        'Authorization': f'Bearer {access_token}',
        'Content-Type': 'application/json'
    }

    #Merge any extra headers (e.g. Prefer header for body content type)
    if extra_headers:
        headers.update(extra_headers)

    url = f'{MS_GRAPH_BASE_ENDPOINT}{endpoint}'

    with httpx.Client(timeout=30.0) as client:
        if method == 'GET':
            response = client.get(url, headers=headers, params=params)
        elif method == 'POST':
            response = client.post(url, headers=headers, json=json_data)
        elif method == 'DELETE':
            response = client.delete(url, headers=headers)
        elif method == 'PATCH':
            response = client.patch(url, headers=headers, json=json_data)

    response.raise_for_status()
    return response.json() if response.text else None
```

Figure 9.3.3.1 make_graph_request helper function

9.3.3.2 Get Email Message Details

Similarly to Gmail, `get_email_message_details()` retrieves the full content of a single email by its message ID. Unlike Gmail where headers are a flat list requiring a search loop, the Graph API returns structured JSON objects, the sender address is accessed directly via nested dictionary keys. The `$select` parameter limits the fields returned by the API to only what is needed, reducing the size of the response.

```
#This function will retrieve the full details of a specific email
def get_email_message_details(access_token, message_id):
    endpoint = f'me/messages/{message_id}'

    #Request full message details
    params = {
        '$select': 'id,subject,from,toRecipients,ccRecipients,body,receivedDateTime,isRead,hasAttachments,importance,flag,conversationId'
    }

    message = make_graph_request(access_token, endpoint, params=params)

    #Extract and format the details similar to Gmail function
    email_details = {
        'id': message.get('id'),
        'subject': message.get('subject', 'No subject'),
        'from': message.get('from', {}).get('emailAddress', {}).get('address', 'Unknown sender'),
        'from_name': message.get('from', {}).get('emailAddress', {}).get('name', ''),
        'to': ', '.join([r.get('emailAddress', {}).get('address', '') for r in message.get('toRecipients', [])]),
        'cc': ', '.join([r.get('emailAddress', {}).get('address', '') for r in message.get('ccRecipients', [])]),
        'body': message.get('body', {}).get('content', '<Text body not available>'),
        'body_type': message.get('body', {}).get('contentType', 'text'),
        'date': message.get('receivedDateTime', 'No date available'),
        'is_read': message.get('isRead', False),
        'has_attachments': message.get('hasAttachments', False),
        'importance': message.get('importance', 'normal'),
        'flagged': message.get('flag', {}).get('flagStatus', 'notFlagged') == 'flagged',
        'conversation_id': message.get('conversationId', message_id)
    }

    return email_details
```

Figure 9.3.3.2 `get_email_message_details` function

9.3.3.3 Sending Email with Attachments

Unlike Gmail which requires the email to be formatted as a MIME message and Base64 encoded, the Graph API accepts a plain JSON body with separate fields for subject, body, and recipients. Attachments are included as a list of objects within the same JSON structure, each with the file content encoded in standard Base64. This makes the Outlook send function considerably simpler to construct than its Gmail equivalent.

```
#This function will send an email with attachments using Microsoft Graph API.
#Unlike Gmail which uses MIME format, Graph API uses JSON structure with separate fields.
def send_email_with_attachment(access_token, to, subject, body, body_type='Text', cc=None, bcc=None, attachment_paths=None):
    #Validate body_type (Graph API uses 'Text' or 'HTML', not lowercase)
    if body_type not in ['Text', 'HTML']:
        raise ValueError("body_type must be either 'Text' or 'HTML'")

    #Helper function to convert email string or list to Graph API recipient format
    def format_recipients(recipients):
        if not recipients:
            return []

        #Convert single string to list
        if isinstance(recipients, str):
            recipients = [recipients]

        #Format each recipient as Graph API expects
        return [{"emailAddress": {"address": email.strip()}} for email in recipients]

    #Build the message structure
    message = {
        "subject": subject,
        "body": {
            "contentType": body_type,
            "content": body
        },
        "toRecipients": format_recipients(to)
    }

    #Add CC recipients if provided
    if cc:
        message["ccRecipients"] = format_recipients(cc)

    #Add BCC recipients if provided
    if bcc:
        message["bccRecipients"] = format_recipients(bcc)

    #Process attachments if provided
    if attachment_paths:
        #Build the request body
        request_body = {
            "message": message,
            "saveToSentItems": True
        }

        #Send the email using Graph API
        endpoint = 'me/sendMail'
        response = make_graph_request(access_token, endpoint, method='POST', json_data=request_body)

    return {"status": "sent", "message": "Email sent successfully"}
```

Figure 9.3.3.3 send_email_with_attachments function

9.3.3.4 Mark as Read

`mark_as_read()` updates the read state of an email. Unlike Gmail which controls read state through label manipulation, Outlook exposes a direct `isRead` boolean field that is updated via a PATCH request.

```
#This function will mark an email as read.
def mark_as_read(access_token, message_id):
    endpoint = f'me/messages/{message_id}'
    json_data = {"isRead": True}

    make_graph_request(access_token, endpoint, method='PATCH', json_data=json_data)

    return f'Message {message_id} marked as read.'
```

Figure 9.3.3.4 `mark_as_read` function

9.3.3.5 Trash Email

`trash_email()` moves an email to Outlook's Deleted Items folder by calling the Graph API's move endpoint with `deleteditems` as the destination. Unlike Gmail which has a dedicated trash endpoint, the Graph API handles trashing by treating it as a folder move operation. The well-known folder name `deleteditems` is used rather than a folder ID, as Graph API accepts these reserved names directly in the URL path.

```
#This function will trash a specific email by moving it to the Deleted Items folder.
def trash_email(access_token, message_id):
    endpoint = f'me/messages/{message_id}/move'

    json_data = {
        'destinationId': 'deleteditems'
    }

    moved_message = make_graph_request(access_token, endpoint, method='POST', json_data=json_data)

    return {
        'id': moved_message.get('id'),
        'message': f'Message {message_id} moved to trash.'
    }
```

Figure 9.3.3.5 `trash_email` function

9.3.4 Teams Integration

The Teams integration follows the same pattern as Outlook, `teams_auth.py` handles OAuth 2.0 authentication and `teams_api.py` contains all API functions. An important design decision was to give Teams its own separate authentication file and its own refresh token file, rather than sharing Outlook's. This is because Teams requires different OAuth scopes (Chat.ReadWrite, Team.ReadBasic.All) compared to Outlook (Mail.ReadWrite), and a single token cannot hold both scope sets simultaneously without conflict.

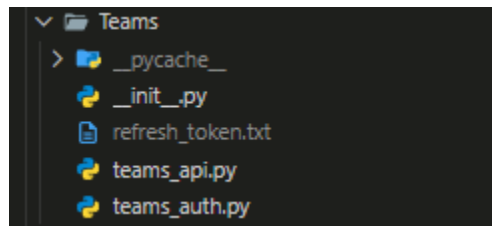


Figure 9.3.4 Teams folder structure

9.3.4.1 Listing Chats

List_chats() function is the GET function for listing the user's Teams chat. The \$expand parameter fetches the last message preview and member list in a single API call, avoiding separate requests per chat.

```
def list_chats(access_token, max_results=10):
    #Get the current user's ID so we can exclude them when naming 1:1 chats.
    me = make_graph_request(access_token, 'me', params={'$select': 'id'})
    my_id = me.get('id', '')

    params = {
        '$top': min(50, max_results),
        '$expand': 'lastMessagePreview,members',
    }

    result = make_graph_request(access_token, 'me/chats', params=params)
    chats = []

    for chat in result.get('value', []):
        preview = chat.get('lastMessagePreview') or {}
        sender_info = preview.get('from') or {}

        #from.user.displayName covers member messages; from.application.displayName covers bot messages.
        sender_name = (
            (sender_info.get('user') or {}).get('displayName')
            or (sender_info.get('application') or {}).get('displayName')
            or 'Unknown'
        )

        members = chat.get('members', [])
        others = [m for m in members if m.get('userId') != my_id]

        > if chat.get('chatType') == 'oneOnOne': ...
        > elif chat.get('topic'): ...
        > else: ...

        chats.append({
            'id': chat['id'],
            'topic': topic,
            'chat_type': chat.get('chatType', 'oneOnOne'),
            'last_sender': sender_name,
            'last_message': preview.get('body', {}).get('content', ''),
            'last_message_time': preview.get('createdDateTime', ''),
            'member_count': len(chat.get('members', [])),
        })

    return chats[:max_results]
```

Figure 9.3.4.1 list_chats function

9.3.4.2 Get Chat Messages

`get_chat_messages()` retrieves messages from a specific Teams chat. The Graph API returns system event messages alongside user messages — join and leave notifications, name changes, and other non-content events — all of which have a `messageType` other than `'message'`. These are filtered out with a simple check so that only actual user messages are returned to the frontend.

```

8 def get_chat_messages(access_token, chat_id, max_results=50):
9     params = {
10         '$top': min(50, max_results),
11         '$orderby': 'createdDateTime desc'
12     }
13
14     result = make_graph_request(access_token, f'chats/{chat_id}/messages', params=params)
15     messages = []
16
17     for msg in result.get('value', []):
18         #Skip system event messages (join/leave notifications, name changes, etc.).
19         #These have messageType != 'message' and no meaningful sender or content.
20         if msg.get('messageType') != 'message':
21             continue
22
23         sender_info = msg.get('from') or {}
24         sender_name = (
25             (sender_info.get('user') or {}).get('displayName')
26             or (sender_info.get('application') or {}).get('displayName')
27             or 'Unknown'
28         )
29
30         body = msg.get('body', {})
31
32         messages.append({
33             'id': msg['id'],
34             'sender_name': sender_name,
35             'content': body.get('content', ''),
36             'content_type': body.get('contentType', 'text'),
37             'created_at': msg.get('createdDateTime', ''),
38             'has_attachments': len(msg.get('attachments', [])) > 0,
39         })
40
41     return messages[:max_results]

```

Figure 9.3.4.2 `get_chat_messages` function

9.3.4.3 Sending Chat Messages

`send_chat_message()` sends a plain-text message to an existing Teams chat. The Graph API returns a 201 Created response with no body when the message is sent successfully, so the function returns `None`. This is different from the Gmail and Outlook send functions, which both return the created message object.

```
def send_chat_message(access_token, chat_id, body):  
    json_data = {  
        'body': {  
            'content': body,  
            'contentType': 'text'  
        }  
    }  
    make_graph_request(access_token, f'chats/{chat_id}/messages', method='POST', json_data=json_data)
```

Figure 9.3.4.3 `send_chat_messages` function

9.3.5 AI Layer

The ai/ folder contains five files that together form the full AI pipeline. embeddings.py handles vector generation, ingestion.py processes and stores emails, retrieval.py performs semantic search, and agents.py contains all six AI agents. Each file has a single, well-defined responsibility.

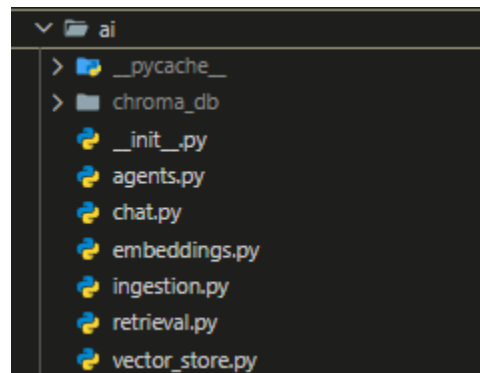


Figure 9.3.5 AI folder structure

9.3.5.1 Embedding text

embed_text() is the single function responsible for converting any piece of text into a vector embedding. It calls the OpenAI text-embedding-3-small model and returns a list of 1536 floating point numbers representing the semantic meaning of the input. This function is called in two places, during ingestion when storing emails, and during retrieval when searching for similar content. Keeping it as a standalone function means both callers always use the same model and produce compatible vectors.

```
def embed_text(text: str) -> list[float]:
    response = _client.embeddings.create(
        model="text-embedding-3-small",
        input=text
    )
    return response.data[0].embedding
```

Figure 9.3.5.1 embed_text function

9.3.5.2 Extracting text

`extract_text()` prepares an email for embedding by combining the subject and body into a single plain-text string. If the email body is HTML, BeautifulSoup strips all the tags and extracts only the readable text. The result is truncated to 8000 characters to stay within the embedding model's token limit.

```
def extract_text(email: dict) -> str:
    subject = email.get("subject", "")
    body = email.get("body", "")
    body_type = email.get("body_type", "text")

    if body_type.lower() == "html":
        body = BeautifulSoup(body, "html.parser").get_text(separator=" ")

    text = f"{subject}\n{body}"
    return text[:8000]
```

Figure 9.3.5.2 `extract_text` function

9.3.5.3 Ingesting Data

This function uses the two `extract_text()` and `embed_text()` function to perform vector embedding and stores it into the vector database. It will return `TRUE` if stored, `FALSE` if skipped which means it is already indexed in the DB or the text is empty, which prevents duplication in the DB.

```
def ingest_email(email: dict, provider: str) -> bool:
    if document_exists(email["id"]):
        return False

    text = extract_text(email)
    if not text.strip():
        return False

    embedding = embed_text(text)
    thread_id = email.get("thread_id") or email.get("conversation_id", "")

    metadata = {
        "provider": provider,
        "message_type": "email",
        "from": email.get("from", ""),
        "date": email.get("date", ""),
        "timestamp": int(datetime.datetime.now().timestamp()),
        "thread_id": thread_id,
        "subject": email.get("subject", "")
    }

    add_document(email["id"], text, embedding, metadata)
    return True
```


9.3.5.4 Context Retrieving

This function is the main function that powers the RAG framework. First, it embeds the query text, searches ChromaDB for the most semantically similar data, and formats the result into a structured plain-text block that can be directly injected into an LLM prompt. Each retrieved email is presented with a numbered header, sender, date, and subject, followed by its full content. Returning empty string when the store is empty ensures all agents degrade gracefully rather than crashing.

```
def retrieve_context(query_text: str, k: int = 3, where: dict | None = None) -> str:
    if not query_text.strip():
        return ""

    query_embedding = embed_text(query_text)

    try:
        results = query_similar(query_embedding, k=k, where=where)
    except Exception:
        #ChromaDB raises if n_results exceeds the number of stored documents.
        #Return empty string so agents degrade gracefully rather than crashing.
        return ""

    #ChromaDB returns results nested in a list – one inner list per query embedding.
    #Since we always query with exactly one embedding, results are always at index [0].
    documents = results.get("documents", [[]])[0]
    metadatas = results.get("metadatas", [[]])[0]

    if not documents:
        return ""

    #Build one formatted section per retrieved email.
    #zip() pairs each document with its metadata dict at the same position.
    #enumerate(start=1) gives a 1-based counter for the section headers.
    sections = []
    for i, (doc, meta) in enumerate(zip(documents, metadatas), start=1):
        sender = meta.get("from", "unknown")
        date = meta.get("date", "unknown")
        subject = meta.get("subject", "")

        header = f"--- Related email {i} ---"
        info = f"From: {sender} | Date: {date} | Subject: {subject}"
        sections.append(f"{header}\n{info}\n{doc}")

    return "\n\n".join(sections)
```

Figure 9.3.5.4 retrieve_context function

9.3.5.5 Email Summarization Agent

This email summarization agent is in charge of producing a 2-3 sentence summary of an email. RAG context is included when available, allowing the model to produce a summary that is informed by related historical emails. temperature=0 is used to keep the output deterministic. max_tokens=150 sets a safe ceiling that comfortably fits 2 to 3 sentences while preventing unnecessarily long outputs.

```
def _build_summarise_prompt(email_text: str, context: str) -> str:
    prompt = f"You are an inbox summarization agent, you must read and summarise the following email in 2 to 3 sentences.\n\n<email>\n{email_text}\n</email>"
    if context:
        prompt += f"\n\nUse the following related emails as additional context:\n\n<context>\n{context}\n</context>"
    return prompt

# Summarises an email in 2 to 3 sentences using GPT-4o-mini.
# context is pre-formatted by retrieve_context() in retrieval.py and passed in by the router.
# temperature=0 keeps the output deterministic – summarisation is a factual task, not a creative one.
# max_tokens=150 is a safe ceiling for 2-3 sentences (~80 words) and prevents runaway output.
def summarize_email(email_text: str, context: str = "") -> str:
    response = _client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "You are an email summarisation assistant. Summarise emails accurately and concisely. Do not add information that is not present in the email."},
            {"role": "user", "content": _build_summarise_prompt(email_text, context)}
        ],
        temperature=0,
        max_tokens=150
    )
    return response.choices[0].message.content.strip()
```

Figure 9.3.5.5 email summarization agent function

9.3.5.6 Email Categorization Agent

This agent classifies an email into one of seven fixed categories. RAG is deliberately not used here since the category of an email is determined entirely by its own content, not by historical context. max_tokens=10 is intentionally tight because the expected response is a single word. .strip().lower() normalises the output in case the model returns variations in casing.

```
# Builds the user message for the categorisation prompt.
# No context block – the category is determined from the email content alone, no RAG needed.
def _build_categorise_prompt(email_text: str) -> str:
    return f"Categorise the following email.\n\n<email>\n{email_text}\n</email>"

# Categorises an email into one of six fixed categories using GPT-4o-mini.
# RAG is not used – the category is self-contained in the email content.
# max_tokens=10 is tight on purpose – we expect exactly one word back.
# .strip().lower() normalises the response in case the model returns "Work" or "WORK".
def categorize_email(email_text: str) -> str:
    response = _client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "You are an email categorisation assistant. Categorise the email into exactly one of these categories: work, personal, family, business, education, health, and other."},
            {"role": "user", "content": _build_categorise_prompt(email_text)}
        ],
        temperature=0,
        max_tokens=10
    )
    return response.choices[0].message.content.strip().lower()
```

Figure 9.3.5.6 email categorization agent function

9.3.5.7 Event Extraction Agent

Event Extraction Agent identifies dates, meetings, deadlines, and action items from an email and returns them as a structured list of JSON objects. A key challenge encountered during development was that the model was resolving relative date references such as "tomorrow" or "next week" against its training data cutoff rather than the actual current date, producing incorrect years. This was fixed by injecting `datetime.date.today().isoformat()` into the system prompt so the model always has access to today's date.

```
def extract_events(email_text: str) -> list[dict]:
    today = datetime.date.today().isoformat()
    response = _client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": f"You are an event extraction assistant. Today's date is {today}. Use this to resolve relative date references."},
            {"role": "user", "content": f"Extract all events from the following email.\n\n<email>\n\n{email_text}\n\n</email>"}
        ],
        temperature=0,
        max_tokens=500,
        response_format={"type": "json_object"}
    )
    result = json.loads(response.choices[0].message.content)
    return result.get("events", [])
```

Figure 9.3.5.7 Event Extraction Agent function

9.3.5.8 Reply Generation Agent

This agent drafts a professional reply to an email. Unlike the other agents, this one uses `temperature=0.3` rather than zero, because a reply should feel natural and human rather than mechanically identical every time. RAG context from past emails is passed in so the model can match the user's typical tone and writing style when responding to familiar senders.

```
def generate_reply(email_text: str, context: str = "") -> str:
    user_message = f"Draft a reply to the following email.\n\n<email>\n\n{email_text}\n\n</email>"
    if context:
        user_message += f"\n\nUse the following past emails to match the tone and writing style of the replies:\n\n<context>\n\n{context}\n\n</context>"
    response = _client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "You are an email reply assistant. Draft a professional and concise reply to the email. I will provide you with the email text and optional context from past emails."},
            {"role": "user", "content": user_message}
        ],
        temperature=0.3,
        max_tokens=300
    )
    return response.choices[0].message.content.strip()
```

Figure 9.3.5.8 Reply Generation Agent function

9.3.5.9 Prioritization Agent

This agent scores an email's urgency on a scale of 1 to 5 and returns both the score and a one-sentence justification. RAG context is used here to identify urgency patterns from past emails with the same sender. The fallback score is 3, a deliberate choice that defaults to normal priority rather than marking everything urgent or low priority when the model is uncertain.

```
def score_priority(email_text: str, context: str = "") -> dict:
    user_message = f"Score the priority of the following email.\n\n<email>\n{email_text}\n</email>"
    if context:
        user_message += f"\n\nUse the following past emails from the same sender as context:\n\n<context>\n{context}\n</context>"
    response = _client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "You are an email priority scoring assistant. Score the urgency of the email on a scale of 1 to 5."},
            {"role": "user", "content": user_message}
        ],
        temperature=0,
        max_tokens=100,
        response_format={"type": "json_object"}
    )
    result = json.loads(response.choices[0].message.content)
    return {"score": result.get("score", 3), "reason": result.get("reason", "")}
```

Figure 9.3.5.9 Prioritization Agent function

9.3.5.10 Spam Detection Agent

This agent determines whether an email is spam and returns a boolean result alongside a confidence score between float number 0 and 1. RAG is not used because spam detection is based entirely on the patterns within the current email. The fallback value for is_spam is False, which is a safer default than True.

```
def is_spam(email_text: str) -> dict:
    response = _client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "You are a spam detection assistant. Determine whether the email is spam or unwanted promotional content."},
            {"role": "user", "content": f"Determine if the following email is spam.\n\n<email>\n{email_text}\n</email>"}
        ],
        temperature=0,
        max_tokens=50,
        response_format={"type": "json_object"}
    )
    result = json.loads(response.choices[0].message.content)
    return {"is_spam": result.get("is_spam", False), "confidence": result.get("confidence", 0.0)}
```

Figure 9.3.5.10 Spam Detection Agent function

9.3.6 Database

The database/ folder contains two files. db.py handles the SQLite connection and table creation, and events_db.py contains all CRUD functions for the calendar events table.

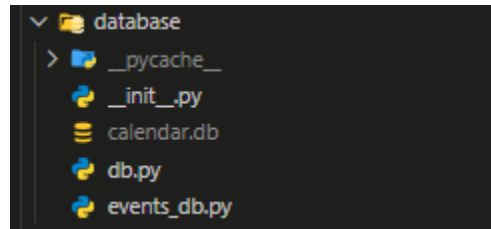


Figure 9.3.6 Database Folder structure

9.3.6.1 DB Initialization

init_db() creates the events table if it does not already exist. It is called once at application startup in main.py, ensuring the table is always present before any calendar requests are made. The CREATE TABLE IF NOT EXISTS statement means this is safe to call every time the server starts without risk of overwriting existing data. event_time and description are nullable because not every extracted event will have a known time or type.

```
def init_db():
    conn = get_db()
    conn.execute('''
        CREATE TABLE IF NOT EXISTS events (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            title         TEXT NOT NULL,
            event_date    TEXT NOT NULL,
            event_time    TEXT,
            description   TEXT,
            provider      TEXT NOT NULL,
            source_email_id TEXT NOT NULL,
            sender_name   TEXT
        )
    ''')
    conn.commit()
    conn.close()
```

Figure 9.3.6.1 DB Initialization function

9.3.6.2 Event Creation

`create_event()` inserts a new calendar event into the database and returns the newly generated row ID. The function is called by the AI router immediately after the event extraction agent returns results, saving each extracted event directly to the database so it appears on the calendar without any additional user action.

```
def create_event(title: str, event_date: str, provider: str, source_email_id: str,
                event_time: str = None, description: str = None, sender_name: str = None) -> int:
    conn = get_db()
    cursor = conn.execute(
        '''INSERT INTO events (title, event_date, event_time, description, provider, source_email_id, sender_name)
        VALUES (?, ?, ?, ?, ?, ?, ?)'''
        (title, event_date, event_time, description, provider, source_email_id, sender_name)
    )
    conn.commit()
    new_id = cursor.lastrowid
    conn.close()
    return new_id
```

Figure 9.3.6.2 Event Creation function

9.3.6.3 Get Events by Month

`get_events_by_month()` retrieves all events for a given year and month, sorted by date and time ascending. Because `event_date` is stored as a YYYY-MM-DD string, a LIKE query with the pattern YYYY-MM-% efficiently filters to the correct month without requiring any date parsing. This is the function called by the calendar page every time the user navigates to a different month.

```
def get_events_by_month(year: int, month: int) -> list[dict]:
    prefix = f'{year}-{month:02d}'
    conn = get_db()
    rows = conn.execute(
        'SELECT * FROM events WHERE event_date LIKE ? ORDER BY event_date ASC, event_time ASC',
        (f'{prefix}-%',)
    ).fetchall()
    conn.close()
    return [dict(row) for row in rows]
```

Figure 9.3.6.3 Get Events by Month function

9.3.6.4 Event Deletion

`delete_event()` removes a single event by its ID and returns a boolean indicating whether a row was actually deleted. Checking `cursor.rowcount` rather than assuming success allows the router to return a proper 404 response if the ID does not exist, rather than silently returning a success status for a deletion that never happened.

```
def delete_event(event_id: int) -> bool:
    conn = get_db()
    cursor = conn.execute('DELETE FROM events WHERE id = ?', (event_id,))
    conn.commit()
    deleted = cursor.rowcount > 0
    conn.close()
    return deleted
```

Figure 9.3.6.4 Event Deletion function

9.4 Frontend

The frontend is a React and TypeScript single-page application structured around a clear separation of concerns.

Pages in `pages/` are responsible for fetching data and rendering full views.

Reusable UI elements live in `components/`.

All data-fetching logic is isolated in custom hooks under `hooks/`.

All Axios API calls are centralised in `services/`.

And global state is managed through React Context in `context/`.

TypeScript interfaces shared across the entire application are defined in a single `types/` file, ensuring consistency between what the backend returns and what the frontend expects. The following subsections cover the key architectural decisions in the frontend.

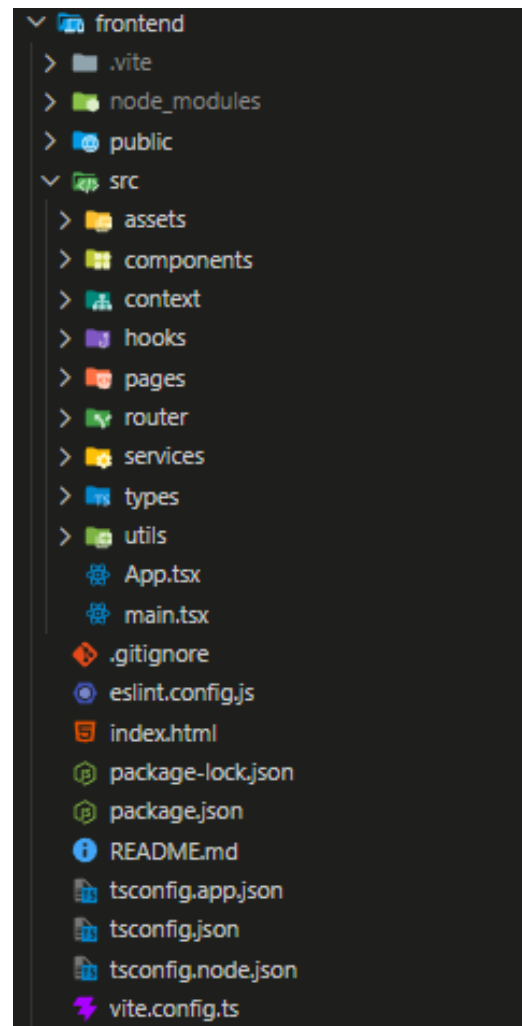


Figure 9.4 Frontend Folder Structure

9.4.1 Routing and Context Provider

9.4.1.1 Router

The router defines all nine routes of the application using React Router v7's `createBrowserRouter`. All routes are nested under the `Layout` component, which renders the persistent sidebar and header so they remain visible regardless of which page is active. The `/email/:provider/:id` route carries both the provider and message ID as URL parameters, allowing the email detail page to know which API to call without any additional state.

```
const router = createBrowserRouter([
  {
    element: <Layout />,
    children: [
      { path: '/', element: <InboxPage /> },
      { path: '/email/:provider/:id', element: <EmailPage /> },
      { path: '/compose', element: <ComposePage /> },
      { path: '/drafts', element: <DraftsPage /> },
      { path: '/chat', element: <ChatPage /> },
      { path: '/sync', element: <SyncPage /> },
      { path: '/teams', element: <TeamsPage /> },
      { path: '/teams/:chatId', element: <TeamsChatPage /> },
      { path: '/calendar', element: <CalendarPage /> },
    ],
  },
]);

export default router;
```

Figure 9.4.1.1 App Router

9.4.1.2 Context Provider

ProviderContext holds the user's active email view mode, gmail, outlook, or unified. Makes it available to every component in the application without prop drilling. The useProvider() hook wraps useContext and throws a descriptive error if called outside the provider, which catches incorrect usage immediately during development rather than producing a silent undefined value.

```
export function ProviderContextProvider({ children }: { children: ReactNode }) {
  const [provider, setProvider] = useState<EmailViewMode>('gmail');

  return (
    <ProviderContext.Provider value={{ provider, setProvider }}>
      {children}
    </ProviderContext.Provider>
  );
}

//useProvider is a custom hook that reads the current provider context.
//Components call useProvider() instead of useContext(ProviderContext) directly.
//Throws an error if used outside of ProviderContextProvider - this catches bugs early.
export function useProvider() {
  const context = useContext(ProviderContext);
  if (!context) {
    throw new Error('useProvider must be used within a ProviderContextProvider');
  }
  return context;
}
```

Figure 9.4.1.2 – 1 Context Provider

```
export type Provider = 'gmail' | 'outlook' | 'teams';
```

Figure 9.4.1.2 – 2 Types of Providers

9.4.2 Type Definitions

Types define the shape of all data flowing through the application. Every TypeScript interface is declared in a single types/ file and imported wherever needed, ensuring the frontend always expects exactly what the backend returns.

Take one as example, EmailSummary is the shared interface used by both Gmail and Outlook email list responses. Provider-specific fields are marked optional with ? so that both providers can return the same interface without type errors.

```
export interface EmailSummary {
  id: string;
  subject: string;
  from: string;
  snippet: string;
  date: string;
  is_read: boolean;
  has_attachments: boolean;
  thread_id?: string; // Gmail only
  from_name?: string; // Outlook only
  provider?: Provider; // Added client-side for unified view
}
```

Figure 9.4.2 EmailSummary interface

9.4.3 Services

Services contain all API call functions. Each service file maps to one backend router, and every function is a one-line arrow function that calls the shared Axios instance.

A single Axios instance is configured in api.ts with the backend base URL and a response interceptor that unwraps the Axios response envelope. Axios wraps every response in an object containing data, status, and headers.

```
const api = axios.create({
  baseURL: 'http://localhost:8000',
});

//This response interceptor unwraps the Axios response envelope. Axios wraps every response
//Since I only need the actual data which is the json from fastapi, this interceptor extracts
api.interceptors.response.use(
  (response) => response.data,
  (error) => {
    //Log the error for debugging
    const message = error.response?.data?.detail || error.message || 'An error occurred';
    console.error('API Error:', message);
    return Promise.reject(error);
  }
);
```

Figure 9.4.3 Axios service instance

9.4.4 Pages

Pages are full-screen route components responsible for fetching data via hooks and passing it down to components for rendering. Pages contain no reusable UI logic.

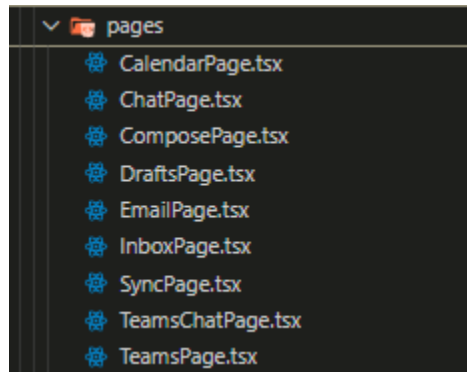


Figure 9.4.4 Pages folder structure

9.4.5 Components

Components receive props and render UI only. They contain no API calls and no data fetching.

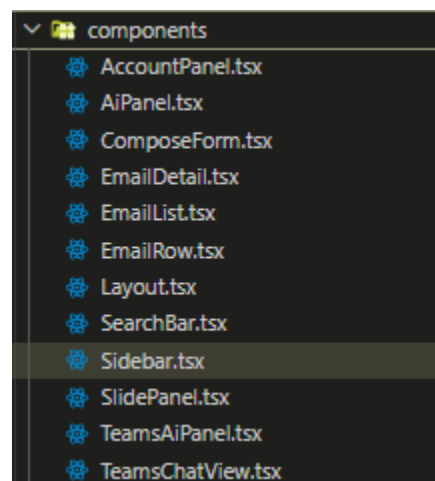


Figure 9.4.5 Components folder structure

9.4.5.1 EmailRow Component

EmailRow renders a single email as a table row displaying the sender, subject snippet, provider tag, and received time. Since Gmail often returns names in the format John Smith <john@example.com>. Clicking the row navigates to /email/:provider/:id, encoding both the provider and message ID in the URL so the email detail page knows which API to call.

```
export default function EmailRow({ email, selectable, selected, onSelect }: EmailRowProps) {
  const navigate = useNavigate();

  const rawSender = email.from_name || email.from || '?';
  const senderName = rawSender.replace(/<[^>*>/g, '').trim() || rawSender;
  const initial = senderName[0].toUpperCase();
  const color = avatarColor(senderName);
  const isUnread = !email.is_read;

  return (
    <div
      onClick={() => navigate(`/email/${email.provider}/${email.id}`)}
      style={{
        display: 'flex',
        alignItems: 'center',
        padding: '12px 16px',
        borderBottom: '1px solid #f3f4f6',
        cursor: 'pointer',
        backgroundColor: '#ffffff',
        transition: 'background-color 120ms ease',
      }}
      onMouseEnter={e => (e.currentTarget.style.backgroundColor = '#f9fafb')}
      onMouseLeave={e => (e.currentTarget.style.backgroundColor = '#ffffff')}
    >
      {selectable && (
        <input
          type="checkbox"
          checked={selected || false}
          onChange={() => onSelect?.(email.id)}
          onClick={e => e.stopPropagation()}
          style={{ cursor: 'pointer', marginRight: 12, flexShrink: 0 }}
        />
      )}
    </div>
  );
}
```

Figure 9.4.5.1 EmailRow Component

10 Project Setup

This section provides the complete guide to setting up and running Intellimail from scratch. It covers all dependencies, third party service configurations and environment variables required before the application can be started.

10.1 Prerequisites

Before beginning, ensure the following are installed in the machine:

- Python 3.11 or Higher
- Node.js 18 or Higher
- Npm
- Git

10.2 Cloning the repository

```
git clone https://github.com/yizhengtang/Intellimail.git
```

```
cd Intellimail
```

10.3 Backend Dependencies

Navigate to the backend directory and install all Python dependencies:

```
cd backend
```

```
pip install -r requirements.txt
```

10.4 Frontend Dependencies

Navigate to the frontend directory to install all Node dependencies:

```
cd frontend
```

```
npm install
```

10.5 Google Cloud Console Setup (Gmail)

To enable Gmail integration, an OAuth2.0 client must be created through the Google Cloud Console:

1. Go to console.cloud.google.com and create a new project, giving it a name. Once created, the **Project ID** is displayed on the project dashboard, copy this value as it is needed for the .env file

2. Navigate to **APIs & Services** → **Library**, search for **Gmail API** and click **Enable**
3. Navigate to **APIs & Services** → **Credentials** and click **Create Credentials** → **OAuth 2.0 Client ID**
4. Set the application type to **Web application**
5. Under **Authorised redirect URIs**, add <http://localhost:8080>
6. Click **Create** and copy the **Client ID** and **Client Secret**

Add the values to backend/.env:

```
#=====
#Google OAuth2 Client Credentials
#=====
GOOGLE_CLIENT_ID=your_google_client_id_here
GOOGLE_CLIENT_SECRET=your_google_client_secret_here
GOOGLE_PROJECT_ID=your_google_project_id_here
GOOGLE_AUTH_URI=https://accounts.google.com/o/oauth2/auth
GOOGLE_TOKEN_URI=https://oauth2.googleapis.com/token
GOOGLE_AUTH_PROVIDER_CERT_URL=https://www.googleapis.com/oauth2/v1/certs
GOOGLE_REDIRECT_URI=http://localhost:8080/
```

Figure 10 – 1 Google OAuth2 Client Credentials

10.6 Azure App Registration Setup (Outlook and Teams)

To enable Outlook and Microsoft Teams integration, an application must be registered through the Microsoft Azure Portal.

1. Go to portal.azure.com and sign in
2. Search for **App Registrations** and click **New Registration**
3. Give the application a name, select **Accounts in any organizational directory and personal Microsoft accounts**, and set the redirect URI to <http://localhost:8090/>
4. Once registered, copy the **Application (client) ID**
5. Navigate to **Certificates & Secrets** → **New Client Secret**, create a secret, and copy the value immediately as it is only shown once

6. Navigate to **API Permissions** → **Add a Permission** → **Microsoft Graph** and add the following delegated permissions: Mail.ReadWrite, Mail.Send, offline_access, User.Read, Chat.Read, Channel.Message.Read.All
7. Click **Grant Admin Consent**

Add the values to backend/.env:

```
#=====
#Microsoft OAuth2 Client Credentials
#=====
MICROSOFT_CLIENT_ID=your_microsoft_client_id_here
MICROSOFT_CLIENT_SECRET=your_microsoft_client_secret_here
MICROSOFT_REDIRECT_URI=http://localhost:8090/
```

Figure 10 – 2 Microsoft OAuth2 Client Credentials

10.7 OpenAI API Key Setup

The OpenAI API key is required to authenticate requests to the OpenAI platform, which powers both the text embedding model used during data ingestion and all six AI agents that perform summarization, categorizations, prioritization, event extraction, reply generation, and spam detection.

1. Go to platform.openai.com and sign in
2. Navigate to API Keys and click Create new secret key
3. Copy the key immediately as it is only shown once

Add the OpenAI API key to backend/.env:

```
#=====
#OpenAI API
#=====
OPENAI_API_KEY=your_openai_api_key_here
```

Figure 10 – 3 OpenAI API key

11 Results

This section presents the final working application, demonstrating each page of the Intellimail interface.

11.1 Gmail Inbox

The Gmail inbox displays emails fetched from the connected Gmail account in a paginated table layout. Each row shows the sender name, subject, a short preview snippet, a colour-coded provider tag, and the received time. Unread emails are displayed in bold. The sidebar on the left shows all Gmail labels with their unread counts, allowing the user to navigate between folders.

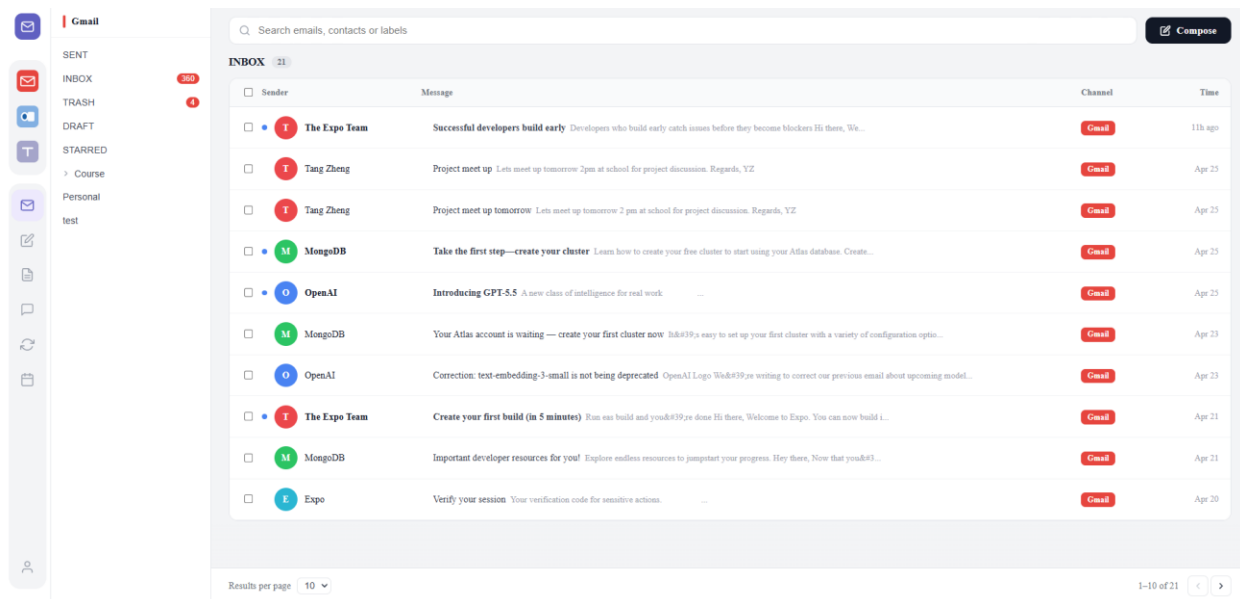


Figure 11.1 Gmail Inbox Page

11.2 Outlook Inbox Page

The Outlook inbox presents the same layout as the Gmail inbox, with emails fetched from the connected Outlook account. The provider tag on each row is coloured blue to distinguish Outlook emails from Gmail. The sidebar displays all Outlook mail folders including Inbox, Sent Items, Drafts, and Deleted Items.

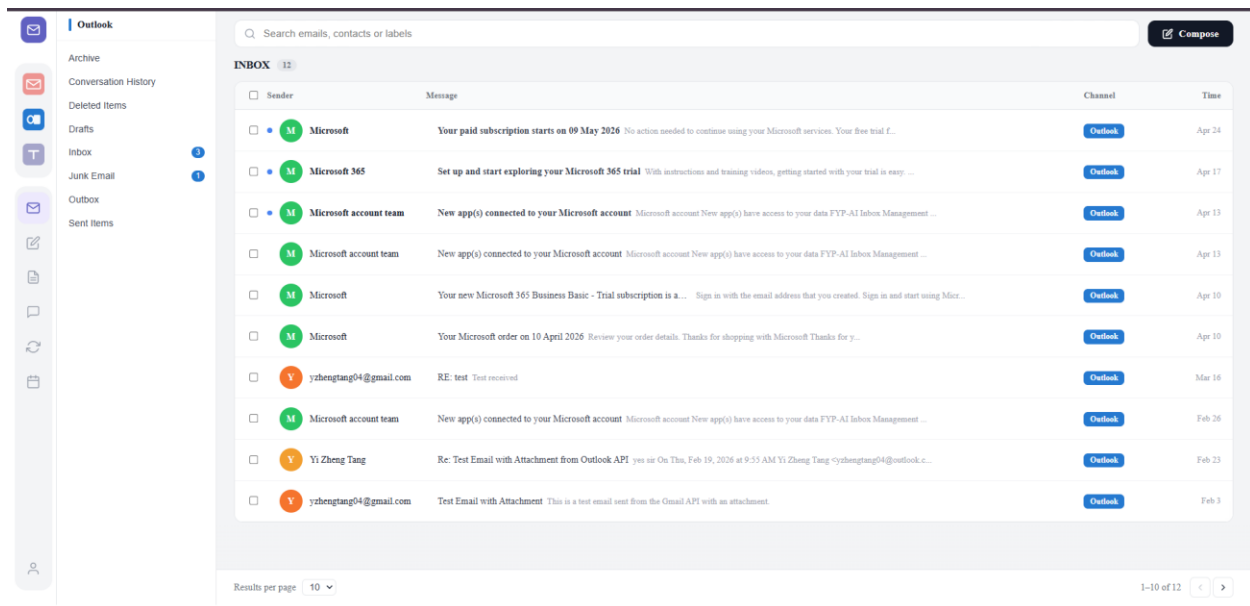


Figure 11.2 Outlook Page

11.3 Email Detail Page

The email detail page displays the full content of a selected email including the sender, recipient, date, and body. From this page, the user can trigger any of the six AI features. The AI-generated result is displayed directly on the page without navigation.

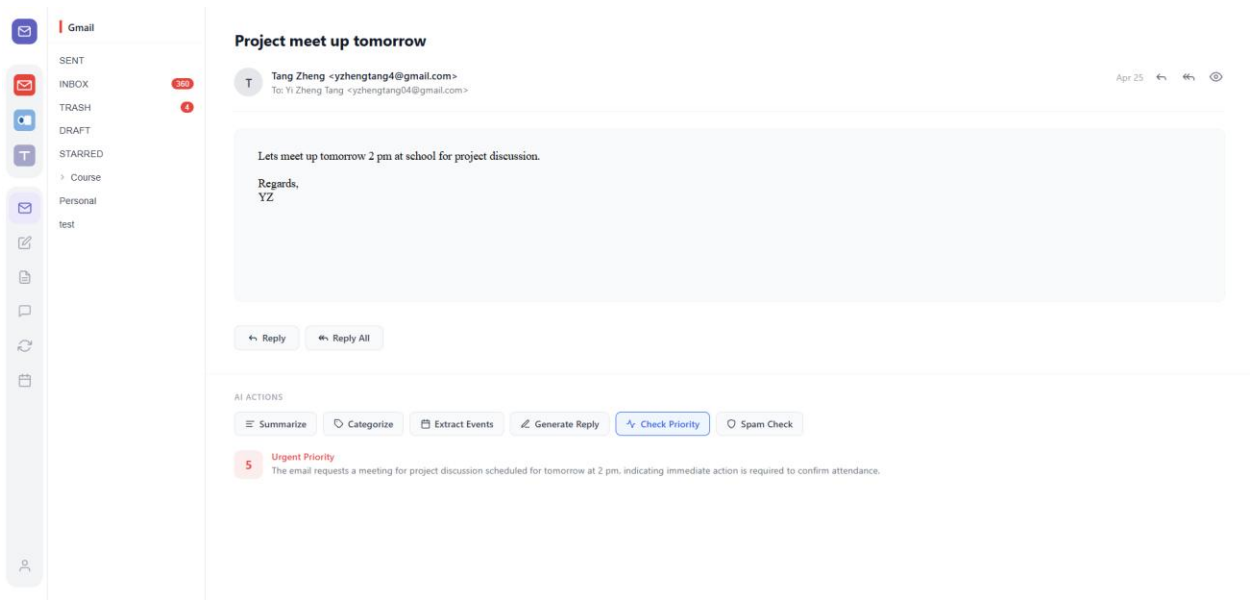


Figure 11.3 Email Detail Page

11.4 Unified Inbox Page

The unified inbox merges emails from both Gmail and Outlook into a single list, sorted by date with the most recent emails at the top. Each email is labelled with its originating provider so the user can identify the source at a glance. If one provider is disconnected, the other loads independently without affecting the view.

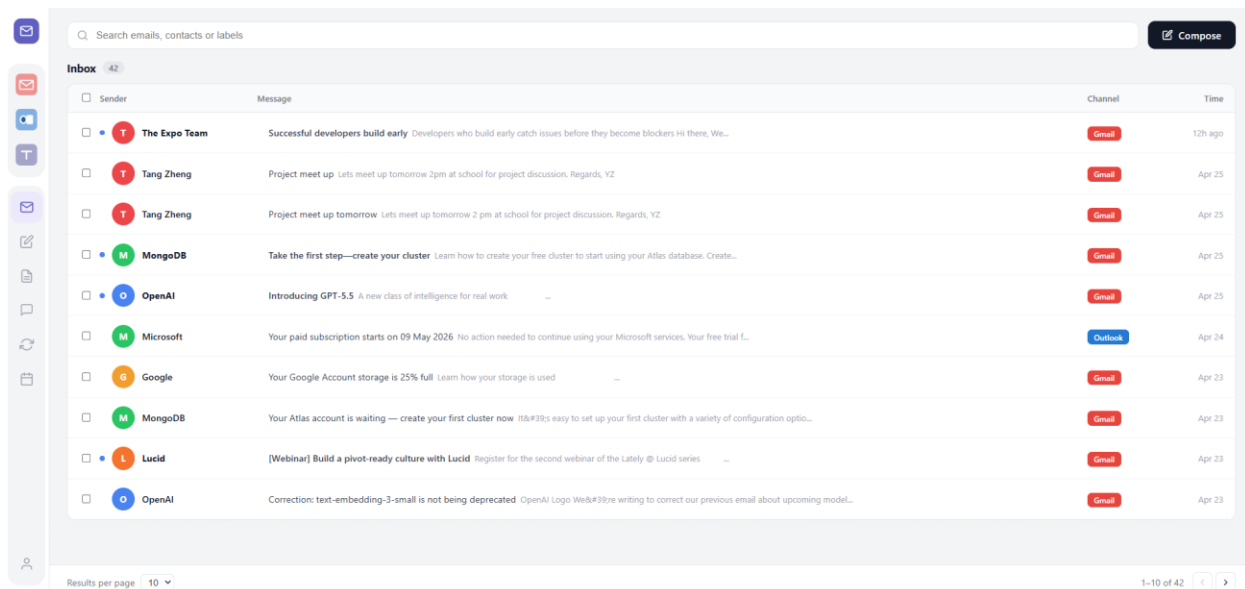


Figure 11.4 Unified Inbox Page

11.5 AI Chat Page

The AI chat page provides a conversational interface where the user can ask questions about their inbox in natural language. When a query is submitted, the system performs a RAG lookup against the vector database, retrieves semantically relevant emails, and passes them as context to GPT-4o-mini to generate an accurate, inbox-grounded response. The full conversation history is maintained throughout the session.

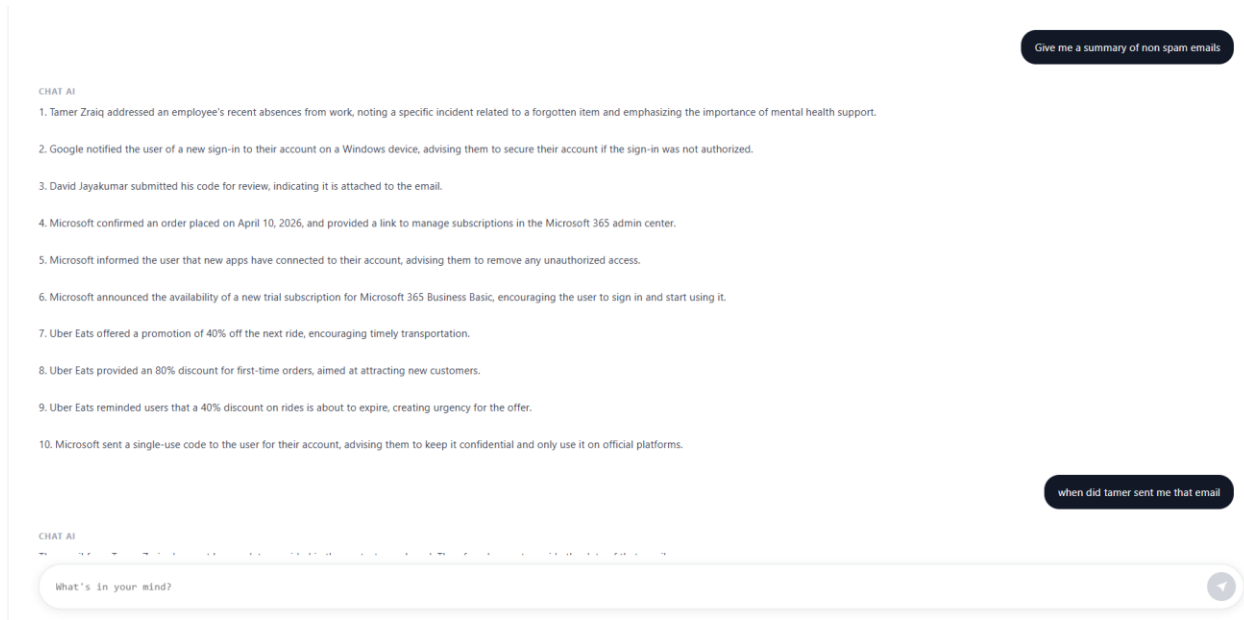


Figure 11.5 AI Chat Page

11.6 Calendar Page

The calendar page displays a monthly grid populated with events automatically extracted from the user's emails. Each event appears as a colour-coded chip on its corresponding date, with the colour indicating the provider the event was extracted from. Clicking an event opens a detail modal showing the event title, date, time, sender name, and event type, with an option to delete the event.

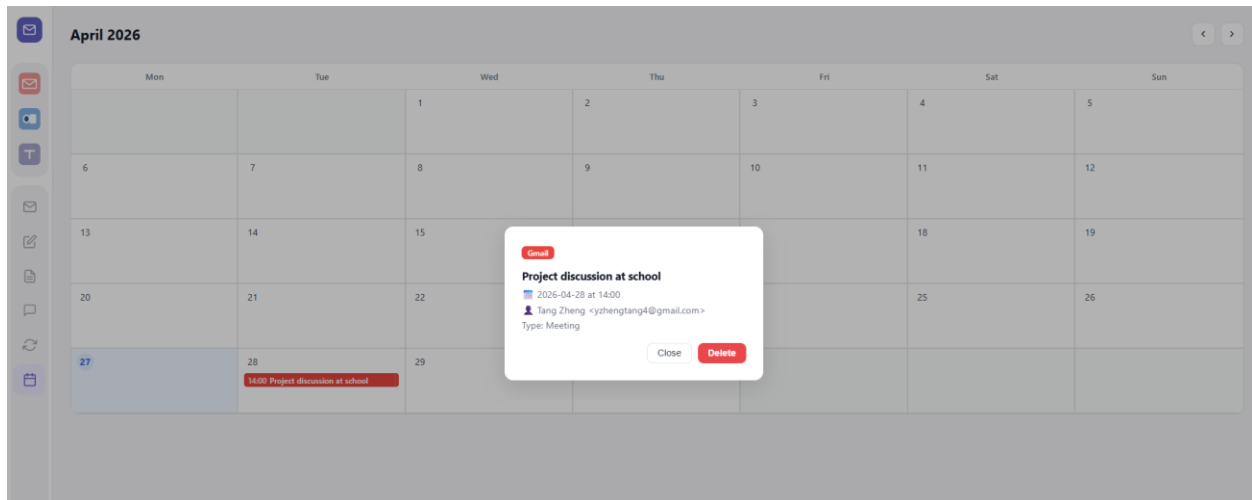


Figure 11.6 Calendar Page

12 Ethics

This project was developed with a commitment to responsible and transparent practice throughout. Three key areas of ethical consideration are addressed below.

12.1 Technology Research and Selection

Before any implementation began, deliberate research was conducted into the suitability of each technology chosen for this project. The selection of OAuth 2.0 for authentication, OpenAI's API for AI capabilities, ChromaDB for vector storage, and SQLite for calendar persistence were each justified on technical grounds. Official documentation, API pricing pages, and academic sources were consulted to verify that each tool was fit for purpose, actively maintained, and appropriate in scope for a Final Year Project. This research process is documented throughout the report in the Technologies Used section, with full IEEE citations provided for every tool introduced.

12.2 Use of AI as a Development Aid

Artificial intelligence tools were used transparently during this project as a coding assistant. Most of the functions and code were written independently first, then brought specific questions to Claude for review or improvement. Any output returned was read critically, understood fully, and verified before being accepted. AI was also used to summarize lengthy technical documentation, such as the Microsoft Graph API schemas and ChromaDB configuration options, which accelerated research without replacing it.

12.3 Data Privacy

The application handles real email data from Gmail and Outlook accounts. OAuth 2.0 tokens are stored locally in files that are excluded from version control via .gitignore. No email content is transmitted to any third party other than the OpenAI API for AI processing. No user data persisted beyond what is necessary for the application to function.

13 Conclusion

Intellimail was conceived as a solution to a genuine problem: the fragmentation of professional communication across multiple email platforms, each with its own interface, workflow, and overhead. The outcome of this project is a fully functional, demonstrable web application that unifies Gmail, Outlook, and Microsoft Teams into a single inbox, augmented by a suite of AI agents that operate directly on the user's own email data.

With the use of proper AI features and frameworks, it leverages the web app in terms of the functionality, giving the user more support and assistance when dealing with their own inboxes.

The result is a working prototype that serves as a proof of concept, demonstrating that a unified, AI-assisted inbox is not only technically feasible but practically usable across multiple real-world email providers. Future development could extend the platform to additional providers such as WhatsApp and Instagram, and explore fine-tuned models trained on domain-specific communication patterns.

14 References

- [1] A. Duke, "Q&A with Gloria Mark, author of 'Attention Span'," *Annie Duke Substack*, [Online]. Available: <https://annieduke.substack.com/p/q-and-a-with-gloria-mark-author-of>
- [2] J. Duffy, "Personal productivity research on task switching," referenced in general productivity literature, 2023.
- [3] Marketplace, "How Slack, email, and Zoom are making us less productive and more overwhelmed," *Marketplace*, Nov. 2023. [Online]. Available: <https://www.marketplace.org/story/2023/11/01/how-slack-email-and-zoom-are-making-us-less-productive-and-more-overwhelmed>
- [4] AZTech IT, "Top tips for managing email overload," *AZTech IT Solutions Blog*. [Online]. Available: <https://www.aztechit.co.uk/blog/top-tips-for-managing-email-overload>
- [5] Shift, "Everything wrong with modern email apps," *Shift Guides*. [Online]. Available: <https://shift.com/guides/email-management/everything-wrong-with-modern-email-apps/>
- [6] Gmelius, "Harnessing AI for email management," *Gmelius Blog*. [Online]. Available: <https://gmelius.com/blog/harnessing-ai-for-email-management>
- [7] Wikipedia, "Unified communications," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Unified_communications
- [8] Lord Neic, "Revolutionizing user interfaces: my vision for a unified chat-driven experience," *Medium*, [Online]. Available: <https://medium.com/@lordNeic/revolutionizing-user-interfaces-my-vision-for-a-unified-chat-driven-ucd-experience-d2e1b14214d3>

- [9] Glean, "AI agents for repetitive low-value tasks," *Glean Perspectives*. [Online]. Available: <https://www.glean.com/perspectives/ai-agents-for-repetitive-low-value-tasks>
- [10] Amazon Web Services, "What is retrieval-augmented generation?," AWS. [Online]. Available: <https://aws.amazon.com/what-is/retrieval-augmented-generation/>
- [11] Nature, "Retrieval-augmented generation study," *Scientific Reports*, 2025. [Online]. Available: <https://www.nature.com/articles/s41598-025-98483-1>
- [12] Glean, "RAG: Retrieval-augmented generation," *Glean Blog*. [Online]. Available: <https://www.glean.com/blog/rag-retrieval-augmented-generation>
- [13] Meta Platforms, Inc., "React – A JavaScript library for building user interfaces," *react.dev*. [Online]. Available: <https://react.dev>
- [14] Microsoft Corporation, "TypeScript: JavaScript with syntax for types," *typescriptlang.org*. [Online]. Available: <https://www.typescriptlang.org>
- [15] E. You, "Vite: Next Generation Frontend Tooling," *vitejs.dev*. [Online]. Available: <https://vitejs.dev>
- [16] Axios contributors, "Axios HTTP Client," *axios-http.com*. [Online]. Available: <https://axios-http.com>
- [17] S. Ramírez, "FastAPI," *fastapi.tiangolo.com*. [Online]. Available: <https://fastapi.tiangolo.com>
- [18] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, Internet Engineering Task Force, Oct. 2012. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc6749>
- [19] Google LLC, "Gmail API Overview," *Google Developers*. [Online]. Available: <https://developers.google.com/gmail/api>

- [20] Microsoft Corporation, "Overview of Microsoft Graph," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/graph/overview>
- [21] OpenAI, "OpenAI API Documentation," *platform.openai.com*. [Online]. Available: <https://platform.openai.com/docs>
- [22] Chroma, "Chroma Documentation," *docs.trychroma.com*. [Online]. Available: <https://docs.trychroma.com>
- [23] D. R. Hipp, "SQLite Home Page," *sqlite.org*. [Online]. Available: <https://www.sqlite.org>